

Optimisation multi-objectif

Xavier Gandibleux

25 Septembre 2026

<http://xgandibleux.github.io/>

Scientific Interests:

- Operations Research:
multi-objective (combinatorial) optimization, (meta)heuristics
- Scientific Computing:
(MO)MILP solvers, Julia programming language

Member of

- Computer Science Department
- LS2N UMR CNRS 6004
- EURO, ROADEF, MCDM, RAMOO, EU/ME

Pour ce séminaire...

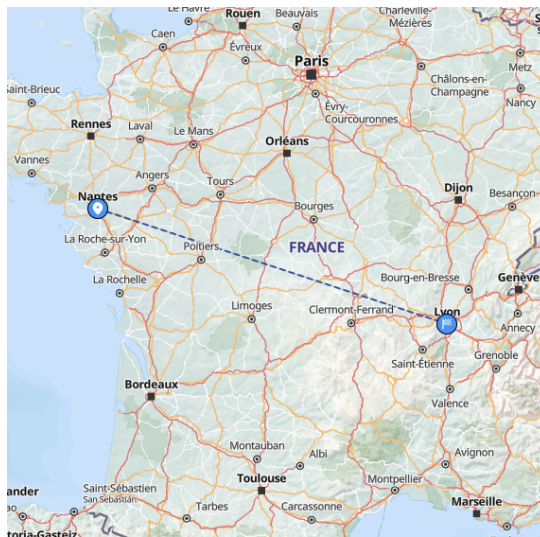
Au programme :

- ▶ Discussion de mise en contexte
- ▶ Notations et définitions de base
- ▶ Quelques algorithmes de base
- ▶ Quels logiciels pour MOMP en 2026
- ▶ Cas d'étude

Ressources en ligne:

https://github.com/xgandibleux/Talks/tree/main/INSA_DISP2026

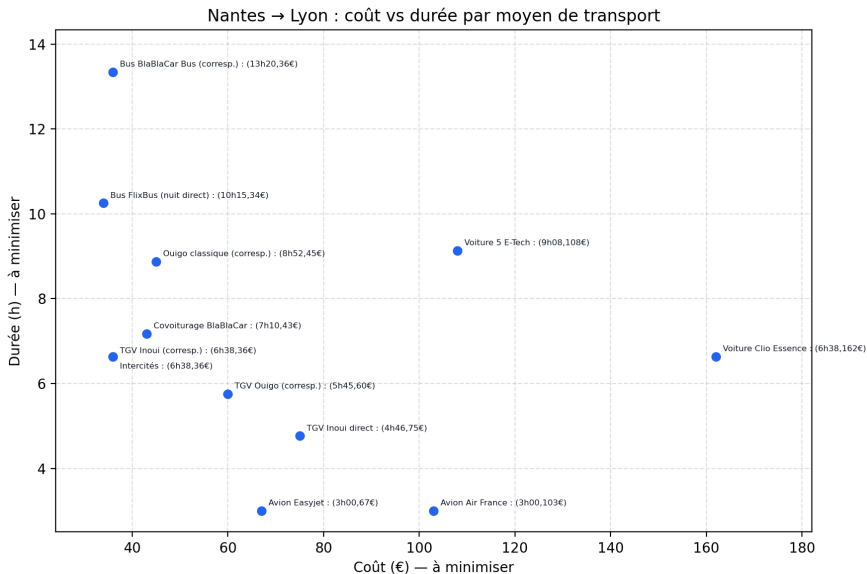
Nantes–Lyon : moyens de transport



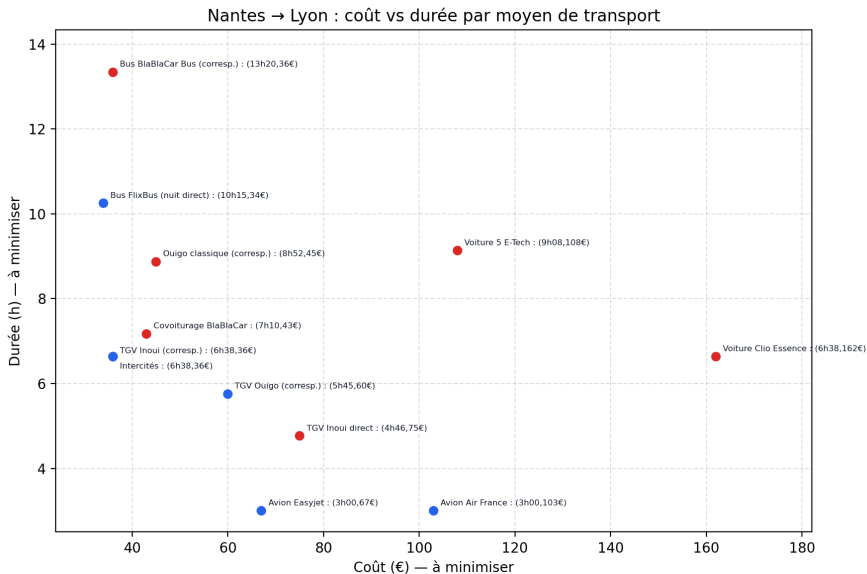
Nantes–Lyon : durée et prix par moyen de transport

Moyen de transport	Durée	Prix
SNCF TGV Inoui direct	4h46	75€
SNCF TGV Inoui (1 correspondance)	6h38	36€
SNCF TGV Ouigo (1 correspondance)	5h45	60€
SNCF Ouigo classique (1 correspondance)	8h52	45€
SNCF Intercités	6h38	36€
Avion Easyjet	3h00	67€
Avion Air France	3h00	103€
Voiture (Clio Essence)	6h38	162€
Voiture (5 E-Tech)	9h08	108€
Covoiturage (BlaBlaCar)	7h10	43€
Bus BlaBlaCar Bus (1 correspondance)	13h20	36€
Bus FlixBus (direct nuit)	10h15	34€

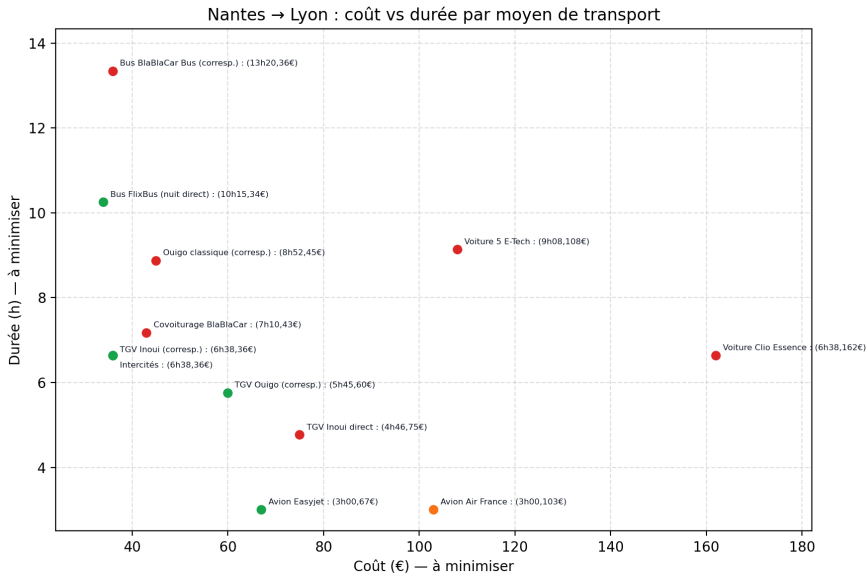
Nantes—Lyon : durée et prix par moyen de transport



Nantes—Lyon : durée et prix par moyen de transport



Nantes—Lyon : durée et prix par moyen de transport



Nantes–Lyon : durée et prix par moyen de transport

Moyen de transport	Durée	Prix
SNCF TGV Inoui direct	4h46	75€
SNCF TGV Inoui (1 correspondance)	6h38	36€
SNCF TGV Ouigo (1 correspondance)	5h45	60€
SNCF Ouigo classique (1 correspondance)	8h52	45€
SNCF Intercités	6h38	36€
Avion Easyjet	3h00	67€
Avion Air France	3h00	103€
Voiture (Clio Essence)	6h38	162€
Voiture (5 E-Tech)	9h08	108€
Covoiturage (BlaBlaCar)	7h10	43€
Bus BlaBlaCar Bus (1 correspondance)	13h20	36€
Bus FlixBus (direct nuit)	10h15	34€

Nantes–Lyon : durée, prix et empreinte carbone

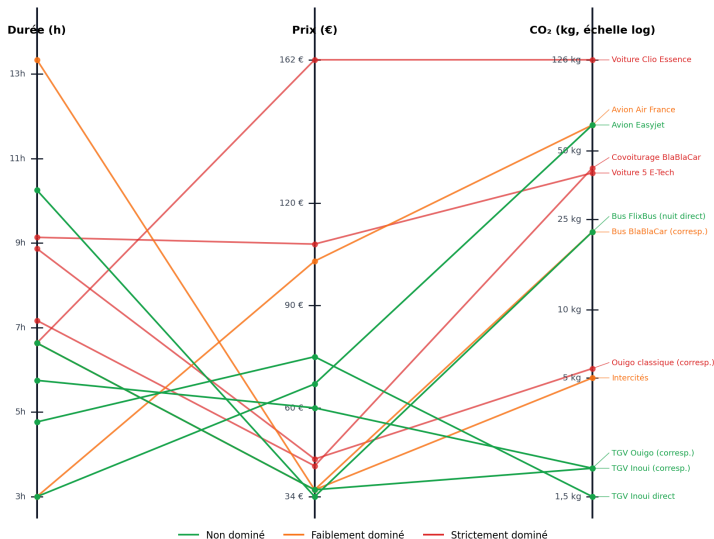
Moyen de transport	Durée	Prix	Empreinte
SNCF TGV Inoui direct	4h46	75€	1,5kg de CO ₂
SNCF TGV Inoui (1 correspondance)	6h38	36€	2kg de CO ₂
SNCF TGV Ouigo (1 correspondance)	5h45	60€	2kg de CO ₂
SNCF Ouigo classique (1 correspondance)	8h52	45€	5,5kg de CO ₂
SNCF Intercités	6h38	36€	5kg de CO ₂
Avion Easyjet	3h00	67€	65 kg de CO ₂
Avion Air France	3h00	103€	65kg de CO ₂
Voiture (Clio Essence)	6h38	162€	126kg de CO ₂
Voiture (5 E-Tech)	9h08	108€	40kg de CO ₂
Covoiturage (BlaBlaCar)	7h10	43€	42kg de CO ₂
Bus BlaBlaCar Bus (1 correspondance)	13h20	36€	22kg de CO ₂
Bus FlixBus (direct nuit)	10h15	34€	22kg de CO ₂

Nantes–Lyon : durée, prix et empreinte carbone

Moyen de transport	Durée	Prix	Empreinte
SNCF TGV Inoui direct	4h46	75€	1,5kg de CO ₂
SNCF TGV Inoui (1 correspondance)	6h38	36€	2kg de CO ₂
SNCF TGV Ouigo (1 correspondance)	5h45	60€	2kg de CO ₂
SNCF Ouigo classique (1 correspondance)	8h52	45€	5,5kg de CO ₂
SNCF Intercités	6h38	36€	5kg de CO ₂
Avion Easyjet	3h00	67€	65 kg de CO ₂
Avion Air France	3h00	103€	65kg de CO ₂
Voiture (Clio Essence)	6h38	162€	126kg de CO ₂
Voiture (5 E-Tech)	9h08	108€	40kg de CO ₂
Covoiturage (BlaBlaCar)	7h10	43€	42kg de CO ₂
Bus BlaBlaCar Bus (1 correspondance)	13h20	36€	22kg de CO ₂
Bus FlixBus (direct nuit)	10h15	34€	22kg de CO ₂

Nantes–Lyon : durée, prix et empreinte carbone

Nantes → Lyon : profils des moyens de transport (3 objectifs à minimiser, meilleur en bas)



Nantes–Lyon : durée, prix et empreinte carbone

Moyen de transport	Durée	Prix	Empreinte	FND/D par ?
SNCF TGV Inoui direct	4h46	75 €	1,5 kg CO ₂	–
SNCF TGV Inoui (1 corr.)	6h38	36 €	2 kg CO ₂	–
SNCF TGV Ouigo (1 corr.)	5h45	60 €	2 kg CO ₂	–
Avion Easyjet	3h00	67 €	65 kg CO ₂	–
Bus FlixBus (direct nuit)	10h15	34 €	22 kg CO ₂	–
SNCF Intercités	6h38	36 €	5 kg CO ₂	TGV Inoui (1 corr.)
Avion Air France	3h00	103 €	65 kg CO ₂	Easyjet
Bus BlaBlaCar Bus (1 corr.)	13h20	36 €	22 kg CO ₂	TGV Inoui (1 corr.)
SNCF Ouigo classique (1 corr.)	8h52	45 €	5,5 kg CO ₂	TGV Inoui (1 corr.)
Voiture (Clio Essence)	6h38	162 €	126 kg CO ₂	TGV Inoui direct
Voiture (5 E-Tech)	9h08	108 €	40 kg CO ₂	TGV Inoui (1 corr.)
Covoiturage (BlaBlaCar)	7h10	43 €	42 kg CO ₂	TGV Inoui (1 corr.)

Donc cinq trajets efficaces

Comment avancer ?

- ▶ faire exprimer les préférences du décideur
- ▶ intégrer ces préférences au sein d'un mécanisme d'agrégation
 - ▶ **méthodes a priori** :
utilisation avant le calcul d'une solution efficace
 - ▶ **méthodes interactives** :
utilisation pendant l'établissement de solutions efficaces
 - ▶ **méthodes a posteriori** :
utilisation après avoir établi toutes les solutions efficaces

Donc cinq trajets efficaces

Comment avancer ?

- ▶ faire exprimer les préférences du décideur
- ▶ intégrer ces préférences au sein d'un mécanisme d'agrégation
 - ▶ méthodes a priori :
utilisation avant le calcul d'une solution efficace
 - ▶ méthodes interactives :
utilisation pendant l'établissement de solutions efficaces
 - ▶ méthodes a posteriori :
utilisation après avoir établi toutes les solutions efficaces

Donc cinq trajets efficaces

Comment avancer ?

- ▶ faire exprimer les préférences du décideur
- ▶ intégrer ces préférences au sein d'un mécanisme d'agrégation
 - ▶ **méthodes a priori** :
utilisation avant le calcul d'une solution efficace
 - ▶ **méthodes interactives** :
utilisation pendant l'établissement de solutions efficaces
 - ▶ **méthodes a posteriori** :
utilisation après avoir établi toutes les solutions efficaces

Quelques exemples de situations

Multiple Objective (Combinatorial) Optimisation MO(C)O:
an efficient modelling tool for many **real life situations**

Optimization challenges have **to balance requirements** stemming from

- ▶ environmental and business concerns,
- ▶ robustness considerations
- ▶ resource efficiency

Example of objectives to be optimised simultaneously, the:

- ▶ maximisation of the classical **profitability**
and
- ▶ minimisation of **CO2 emission** in logistics
(vehicle routing, plant location, etc.),
- ▶ maximisation of **robustness of a timetable** in a transportation system
(trains, buses, etc.),
- ▶ minimisation of **energy consumption** in a production system
(servers, railway, etc.)
- ▶ ...

Quelques exemples de situations

Multiple Objective (Combinatorial) Optimisation MO(C)O:
an efficient modelling tool for many **real life situations**

Optimization challenges have **to balance requirements** stemming from

- ▶ environmental and business concerns,
- ▶ robustness considerations
- ▶ resource efficiency

Example of objectives to be optimised simultaneously, the:

- ▶ maximisation of the classical **profitability** and
- ▶ minimisation of **CO2 emission** in logistics (vehicle routing, plant location, etc.),
- ▶ maximisation of **robustness of a timetable** in a transportation system (trains, buses, etc.),
- ▶ minimisation of **energy consumption** in a production system (servers, railway, etc.)
- ▶ ...

Quelques exemples de situations

Multiple Objective (Combinatorial) Optimisation MO(C)O:
an efficient modelling tool for many **real life situations**

Optimization challenges have **to balance requirements** stemming from

- ▶ environmental and business concerns,
- ▶ robustness considerations
- ▶ resource efficiency

Example of objectives to be optimised simultaneously, the:

- ▶ maximisation of the classical **profitability**
and
- ▶ minimisation of **CO2 emission** in logistics
(vehicle routing, plant location, etc.),
- ▶ maximisation of **robustness of a timetable** in a transportation system
(trains, buses, etc.),
- ▶ minimisation of **energy consumption** in a production system
(servers, railway, etc.)
- ▶ ...

Quelques exemples de situations

MOCO problems are difficult problems to optimize because they present several (possibly all) of the following characteristics:

- ▶ very bad theoretical complexity
- ▶ large-size datasets representing a real life situation
- ▶ computing time limited for producing a solution
- ▶ data often not fully determinist

→ Des **algorithmes exacts de plus en plus compétitifs et sinon des métaheuristiques multiobjectif** (notamment en présence de cas réels)

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
- ▶ exact solving methods are limited to problems with **only two objectives**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
- ▶ exact solving methods are limited to problems with **only two objectives**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
- ▶ exact solving methods are limited to problems with **only two objectives**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
⇒ **wrong in general**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
⇒ **wrong in general**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
⇒ **wrong in general**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
⇒ **wrong in theory, wrong in practice**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
⇒ **wrong in general**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
⇒ **wrong in theory, wrong in practice**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.

Quelques fausses bonnes idées

- ▶ building a **weighted sum** of the objectives and solving the single objective problem
⇒ **wrong approach in most general case**
- ▶ only **evolutionary algorithms** are efficient for solving a “difficult” combinatorial optimization problem
⇒ **wrong for more and more structured problems**
- ▶ exact solving methods are limited to problems with **only two objectives**
⇒ **wrong in general**
- ▶ multi-objective optimization starts with problems handling **at least 3 objectives**
⇒ **wrong in theory, wrong in practice**
- ▶ from **1 to 2** objectives, from **2 to 3** objectives, etc.
⇒ **wrong in general**

References and Community

References



- Ralph E. Steuer. *Multiple Criteria Optimization: Theory, Computation, and Application*. Wiley Series in Probability and Statistics, 1986.

- **Matthias Ehrgott. *Multicriteria Optimization*. Springer Berlin Heidelberg, 2005.**



- Matthias Ehrgott, Xavier Gandibleux. *Multiple Criteria Optimization: State of the Art Annotated Bibliographic Surveys*. International Series in Operations Research & Management Science, Springer New York, 2002.



- Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons Inc, 2001

- Carlos Coello Coello, Gary Lamont, David van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Springer, 2e edition, 2007.



- Matthias Ehrgott, Carlos M. Fonseca, Xavier Gandibleux, Jin-Kao Hao, Marc Sevaux. *Evolutionary Multi-Criterion Optimization : 5th International Conference, EMO 2009, Nantes, France, April 7-10, 2009, Proceedings*. Lecture Notes in Computer Science, volume 5467, Springer Berlin Heidelberg, 2009.

References



- Ralph E. Steuer. *Multiple Criteria Optimization: Theory, Computation, and Application*. Wiley Series in Probability and Statistics, 1986.

- **Matthias Ehrgott. *Multicriteria Optimization*. Springer Berlin Heidelberg, 2005.**



- Matthias Ehrgott, Xavier Gandibleux. *Multiple Criteria Optimization: State of the Art Annotated Bibliographic Surveys*. International Series in Operations Research & Management Science, Springer New York, 2002.



- **Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons Inc, 2001**

- Carlos Coello Coello, Gary Lamont, David van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Springer, 2e edition, 2007.



- Matthias Ehrgott, Carlos M. Fonseca, Xavier Gandibleux, Jin-Kao Hao, Marc Sevaux. *Evolutionary Multi-Criterion Optimization : 5th International Conference, EMO 2009, Nantes, France, April 7-10, 2009, Proceedings*. Lecture Notes in Computer Science, volume 5467, Springer Berlin Heidelberg, 2009.

References

MOMP

- Xavier Gandibleux: Native Multi-Objective Mathematical Programming Software. Chapter 15 in *Handbook on Multi-objective Combinatorial Optimisation* (M. Ehrgott, J. Figueira, Ö. Karsu, L. Paquete Edts.). Springer, 2026 (to appear).
- Oscar Dowson, Xavier Gandibleux, Gökhan Kof: MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems. *INFORMS Journal of Computing*, 2026.

MOMH

- Michael T. M. Emmerich, André H. Deutz: A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing* 17(3): 585-609, 2018.
- Antonio Benítez-Hidalgo, Antonio J. Nebro, José García-Nieto, Izaskun Oregi, Javier Del Ser: jMetalPy: A Python framework for multi-objective optimization with metaheuristics. *Swarm and Evolutionary Computation* 51, 2019.

References

MOMP

- Xavier Gandibleux: Native Multi-Objective Mathematical Programming Software. Chapter 15 in *Handbook on Multi-objective Combinatorial Optimisation* (M. Ehrgott, J. Figueira, Ö. Karsu, L. Paquete Edts.). Springer, 2026 (to appear).
- Oscar Dowson, Xavier Gandibleux, Gökhan Kof: MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems. *INFORMS Journal of Computing*, 2026.

MOMH

- Michael T. M. Emmerich, André H. Deutz: A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing* 17(3): 585-609, 2018.
- Antonio Benítez-Hidalgo, Antonio J. Nebro, José García-Nieto, Izaskun Oregi, Javier Del Ser: jMetalPy: A Python framework for multi-objective optimization with metaheuristics. *Swarm and Evolutionary Computation* 51, 2019.

Community

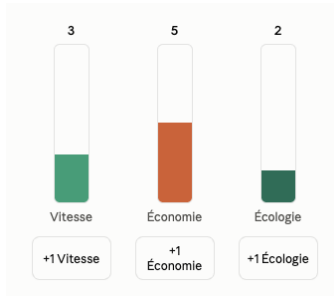
- MCDM
<https://www.mcdmsociety.org/>
- MOPGP
<https://mopgp.org/>
- RAMOO
<https://moo.univie.ac.at/>
- EMO
<http://www.emo-online.org/>
- ROADEF
<http://www.roadef.org>
- GDR CNRS Recherche Opérationnelle et Décision
<http://gdrro.lip6.fr/>

Community

- MCDM
<https://www.mcdmsociety.org/>
- MOPGP
<https://mopgp.org/>
- RAMOO
<https://moo.univie.ac.at/>
- EMO
<http://www.emo-online.org/>
- ROADEF
<http://www.roadef.org>
- GDR CNRS Recherche Opérationnelle et Décision
<http://gdrro.lip6.fr/>

Multi-Objective Optimization

Basics



General definitions
in
Multi-Objective Optimization (MOO)

Multi-Objective Optimization Problem (MOP)

Common usual notations

► MOP:
$$\begin{array}{ll} \text{"min" } z(x) \\ \text{subject to } g_j(x) \leq 0 & j = 1, \dots, m \\ x \in \mathbb{R}^n \end{array}$$

$$\begin{array}{lll} x \in \mathbb{R}^n & \longrightarrow & n \text{ variables} \quad i = 1, \dots, n \\ g \in \mathbb{R}^n \rightarrow \mathbb{R}^m & \longrightarrow & m \text{ constraints} \quad j = 1, \dots, m \\ z \in \mathbb{R}^n \rightarrow \mathbb{R}^p & \longrightarrow & p \text{ objective functions } k = 1, \dots, p \end{array}$$

- $X = \{x \in \mathbb{R}^n : g_j(x) \leq 0\}$ feasible set in decision space $\mathbb{R}^n$
 $x \in \mathbb{R}^n$ is called a *solution*
 $x \in X$ a **feasible solution**
- $Y = z(X) = \{z(x) : x \in X\}$ feasible set in objective space $\mathbb{R}^p$
(also called *outcome set*)
 $y \in \mathbb{R}^p$ is called a *point* (or *vector*)
 $y \in Y$ a **feasible point**

Multi-Objective Optimization Problem, MOP

Definitions of Optimality

"min" has to be stated.

Two definitions of optimality are considered:

- ▶ **Lexicographic optimality:**

There is a ranking among objectives

Example:

$$\operatorname{lexmin}_{x \in X} \{z^2(x), z^3(x), z^1(x)\}$$

- ▶ **Efficiency and nondominance:**

No objective is more significant than any other one

Example:

$$\operatorname{vmin}_{x \in X} \{z^1(x), z^2(x), z^3(x)\}$$

Multi-Objective Optimization Problem, MOP

Definitions of Optimality

"min" has to be stated.

Two definitions of optimality are considered:

- ▶ **Lexicographic optimality:**

There is a ranking among objectives

Example:

$$\operatorname{lexmin}_{x \in X} \{z^2(x), z^3(x), z^1(x)\}$$

- ▶ **Efficiency and nondominance:**

No objective is more significant than any other one

Example:

$$\operatorname{vmin}_{x \in X} \{z^1(x), z^2(x), z^3(x)\}$$

Multi-Objective Optimization Problem, MOP

Lexicographic Optimality

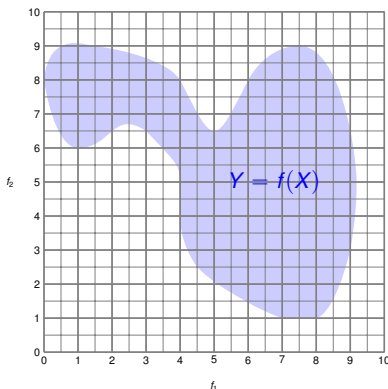
Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$

If $y^1 \neq y^2$, let $k^* = \min\{k | y_k^1 \neq y_k^2\}$

y^1 **lex optimal** y^2

$y^1 \leq_{lex} y^2 \Leftrightarrow y^1 = y^2 \text{ or } y_{k^*}^1 < y_{k^*}^2$

- ▶ Lexicographic optimality (1)
 $z(\hat{x}) \leq_{lex} z(x)$ for all $x \in X$
- ▶ Lexicographic optimality (2)
 $z^\pi(\hat{x}) \leq_{lex} z^\pi(x)$ for all $x \in X$
and some permutation z^π of
 $(z_1, \dots, z_p)$



Multi-Objective Optimization Problem, MOP

Lexicographic Optimality

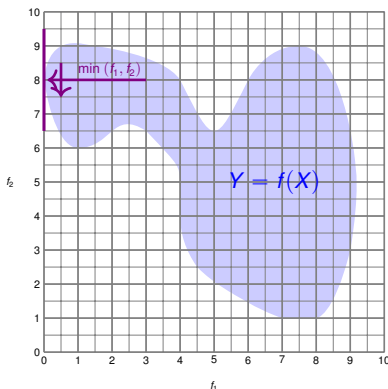
Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$

If $y^1 \neq y^2$, let $k^* = \min\{k | y_k^1 \neq y_k^2\}$

y^1 **lex optimal** y^2

$y^1 \leq_{lex} y^2 \Leftrightarrow y^1 = y^2 \text{ or } y_{k^*}^1 < y_{k^*}^2$

- ▶ Lexicographic optimality (1)
 $z(\hat{x}) \leq_{lex} z(x)$ for all $x \in X$
- ▶ Lexicographic optimality (2)
 $z^\pi(\hat{x}) \leq_{lex} z^\pi(x)$ for all $x \in X$
and some permutation z^π of
 $(z_1, \dots, z_p)$



Multi-Objective Optimization Problem, MOP

Lexicographic Optimality

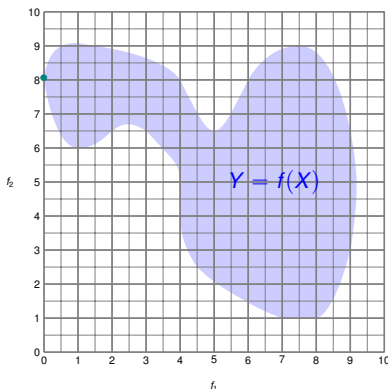
Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$

If $y^1 \neq y^2$, let $k^* = \min\{k | y_k^1 \neq y_k^2\}$

y^1 **lex optimal** y^2

$y^1 \leq_{lex} y^2 \Leftrightarrow y^1 = y^2 \text{ or } y_{k^*}^1 < y_{k^*}^2$

- ▶ Lexicographic optimality (1)
 $z(\hat{x}) \leq_{lex} z(x)$ for all $x \in X$
- ▶ Lexicographic optimality (2)
 $z^\pi(\hat{x}) \leq_{lex} z^\pi(x)$ for all $x \in X$
and some permutation z^π of
 $(z_1, \dots, z_p)$



Multi-Objective Optimization Problem, MOP

Lexicographic Optimality

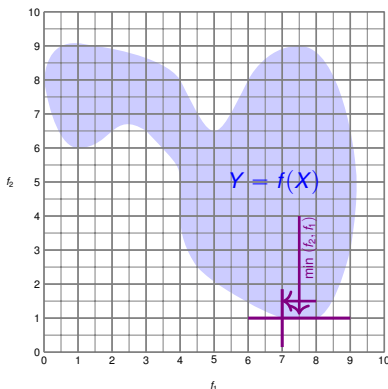
Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$

If $y^1 \neq y^2$, let $k^* = \min\{k | y_k^1 \neq y_k^2\}$

y^1 **lex optimal** y^2

$y^1 \leq_{lex} y^2 \Leftrightarrow y^1 = y^2 \text{ or } y_{k^*}^1 < y_{k^*}^2$

- ▶ Lexicographic optimality (1)
 $z(\hat{x}) \leq_{lex} z(x)$ for all $x \in X$
- ▶ Lexicographic optimality (2)
 $z^\pi(\hat{x}) \leq_{lex} z^\pi(x)$ for all $x \in X$
and some permutation z^π of
 $(z_1, \dots, z_p)$



Multi-Objective Optimization Problem, MOP

Lexicographic Optimality

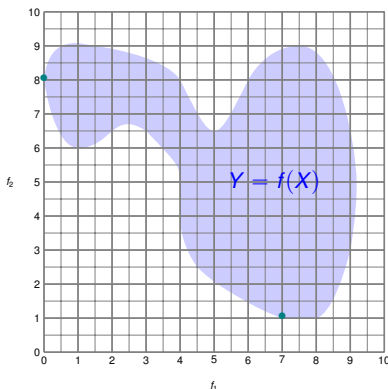
Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$

If $y^1 \neq y^2$, let $k^* = \min\{k | y_k^1 \neq y_k^2\}$

y^1 **lex optimal** y^2

$y^1 \leq_{lex} y^2 \Leftrightarrow y^1 = y^2 \text{ or } y_{k^*}^1 < y_{k^*}^2$

- ▶ Lexicographic optimality (1)
 $z(\hat{x}) \leq_{lex} z(x)$ for all $x \in X$
- ▶ Lexicographic optimality (2)
 $z^\pi(\hat{x}) \leq_{lex} z^\pi(x)$ for all $x \in X$
and some permutation z^π of
 $(z_1, \dots, z_p)$

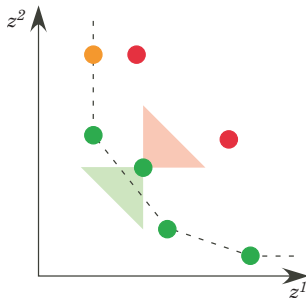


Multi-Objective Optimization Problem, MOP

(Weakly) Efficient Solutions

Let $y^1, y^2 \in \mathbb{R}^p$ with $p > 1$:

- ▶ $y^1 \leq y^2 \Leftrightarrow y_k^1 \leq y_k^2$ for $k = 1, \dots, p$
 y^1 **weakly dominates** y^2
- ▶ $y^1 < y^2 \Leftrightarrow y_k^1 < y_k^2$ for $k = 1, \dots, p$
 y^1 **strictly dominates** y^2
- ▶ $y^1 \leq y^2 \Leftrightarrow y^1 \leq y^2$ and $y^1 \neq y^2$
 y^1 **dominates** y^2
- ▶ $\mathbb{R}_{\leq}^p = \{y \in \mathbb{R}^p : y \leq 0\}$
- ▶ $\mathbb{R}_{\geq}^p = \{y \in \mathbb{R}^p : y \geq 0\}$
- ▶ $\mathbb{R}_{>}^p = \{y \in \mathbb{R}^p : y > 0\}$



Multi-Objective Optimization Problem, MOP

(Weakly) Efficient Solutions

- ▶ **Weakly efficient** solutions X_{wE} :

$\hat{x} \in X$ such that
there is no $x \in X$ with $z(x) < z(\hat{x})$
 $z(\hat{x})$ is **weakly nondominated**

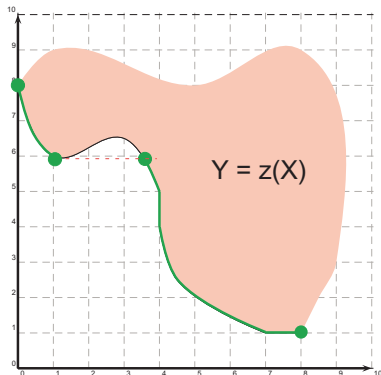
$Y_{wN} := z(X_{wE})$: the set of weakly
nondominated points

- ▶ **Efficient** solutions X_E :

$\hat{x} \in X$ such that
there is no $x \in X$ with $z(x) \leq z(\hat{x})$
 $z(\hat{x})$ is **nondominated**

$Y_N := z(X_E)$: the set of nondominated
points

- ▶ $X_E \subseteq X_{wE}$
 $Y_N \subseteq Y_{wN}$



Multi-Objective Optimization Problem, MOP

(Weakly) Efficient Solutions

- ▶ **Weakly efficient** solutions X_{wE} :

$\hat{x} \in X$ such that
there is no $x \in X$ with $z(x) < z(\hat{x})$
 $z(\hat{x})$ is **weakly nondominated**

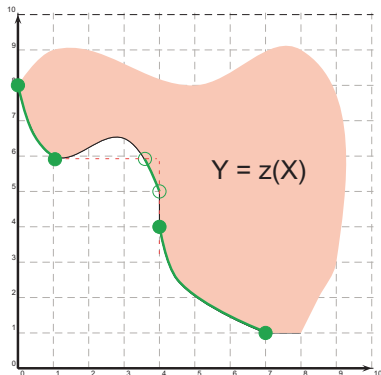
$Y_{wN} := z(X_{wE})$: the set of weakly
nondominated points

- ▶ **Efficient** solutions X_E :

$\hat{x} \in X$ such that
there is no $x \in X$ with $z(x) \leq z(\hat{x})$
 $z(\hat{x})$ is **nondominated**

$Y_N := z(X_E)$: the set of nondominated
points

- ▶ $X_E \subseteq X_{wE}$
 $Y_N \subseteq Y_{wN}$



Classification of Efficient Sets (1/3)

[minimal / maximal] Complete set

$x^1, x^2 \in X$ are equivalent if $z(x^1) = z(x^2)$

- ▶ **Complete set:**

Set of efficient solutions X_E such that for all $y \in Y_N$ there is $x \in X_E$ with $z(x) = y$

- ▶ **Minimal complete set** (Hansen 1979):

Any complete set X_{E_m} without equivalent solutions

- ▶ **Maximal complete set:**

The complete set X_{E_M} including all equivalent solutions

Classification of Efficient Sets (1/3)

[minimal / maximal] Complete set

$x^1, x^2 \in X$ are equivalent if $z(x^1) = z(x^2)$

- ▶ **Complete set:**

Set of efficient solutions X_E such that for all $y \in Y_N$ there is $x \in X_E$ with $z(x) = y$

- ▶ **Minimal complete set** (Hansen 1979):

Any complete set X_{E_m} without equivalent solutions

- ▶ **Maximal complete set:**

The complete set X_{E_M} including all equivalent solutions

Classification of Efficient Sets (1/3)

[minimal / maximal] Complete set

$x^1, x^2 \in X$ are equivalent if $z(x^1) = z(x^2)$

- ▶ **Complete set:**

Set of efficient solutions X_E such that for all $y \in Y_N$ there is $x \in X_E$ with $z(x) = y$

- ▶ **Minimal complete set** (Hansen 1979):

Any complete set X_{E_m} without equivalent solutions

- ▶ **Maximal complete set:**

The complete set X_{E_M} including all equivalent solutions

Classification of Efficient Solutions (2/3)

Supported and nonsupported

- ▶ **Supported efficient solutions:**

There is $\lambda \in \mathbb{R}_{>}^p$ with

$$\lambda^T z(\hat{x}) \leq \lambda^T z(x) \text{ for all } x \in X$$

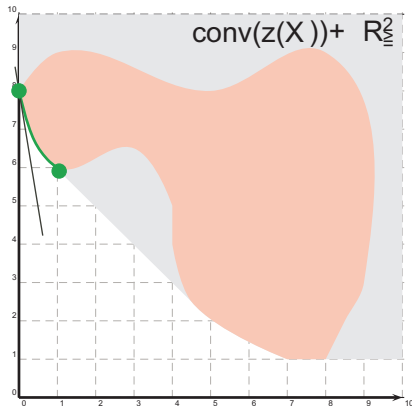
Y_{SN} : The set of supported points

- ▶ $z(\hat{x})$ is **extreme point** of $\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN1}$
- ▶ $z(\hat{x})$ is **in relative interior** of face of $\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN2}$

- ▶ **Nonsupported efficient solutions:**

$z(\hat{x})$ is **in interior** of

$$\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{NN}$$



Classification of Efficient Solutions (2/3)

- ▶ **Supported efficient solutions:**
 There is $\lambda \in \mathbb{R}_{\geq}^p$ with
 $\lambda^T z(\hat{x}) \leq \lambda^T z(x)$ for all $x \in X$
 Y_{SN} : The set of supported points
 - ▶ $z(\hat{x})$ is **extreme point** of
 $\text{conv } Y + \mathbb{R}_{\leq}^p \rightarrow Y_{SN1}$
 - ▶ $z(\hat{x})$ is **in relative interior** of
 face of $\text{conv } Y + \mathbb{R}_{\leq}^p \rightarrow Y_{SN2}$
- ▶ **Not supported efficient solutions:**
 $z(\hat{x})$ is **in interior** of
 $\text{conv } Y + \mathbb{R}_{\leq}^p \rightarrow Y_{NN}$

Classification of Efficient Solutions (2/3)

Supported and nonsupported

- ▶ **Supported efficient solutions:**

There is $\lambda \in \mathbb{R}_{>}^p$ with

$\lambda^T z(\hat{x}) \leq \lambda^T z(x)$ for all $x \in X$

Y_{SN} : The set of supported points

- ▶ $z(\hat{x})$ is **extreme point** of

$\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN1}$

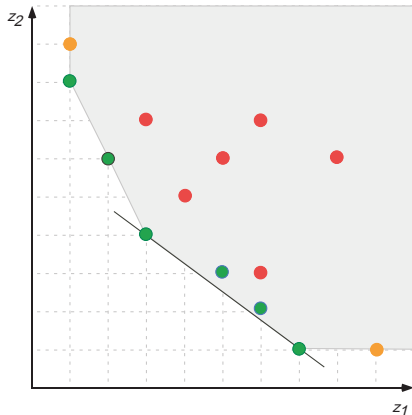
- ▶ $z(\hat{x})$ is **in relative interior** of

face of $\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN2}$

- ▶ **Nonsupported efficient solutions:**

$z(\hat{x})$ is **in interior** of

$\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{NN}$



Classification of Efficient Solutions (2/3)

Supported and nonsupported

- ▶ **Supported efficient solutions:**

There is $\lambda \in \mathbb{R}_{>}^p$ with

$\lambda^T z(\hat{x}) \leq \lambda^T z(x)$ for all $x \in X$

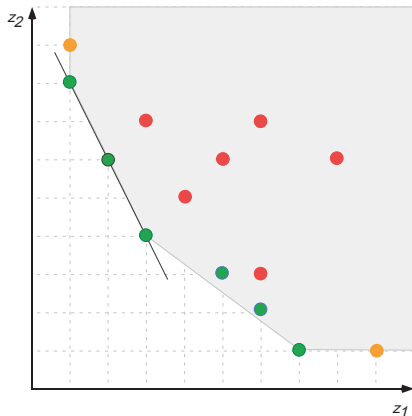
Y_{SN} : The set of supported points

- ▶ $z(\hat{x})$ is **extreme point** of $\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN1}$
- ▶ $z(\hat{x})$ is **in relative interior** of face of $\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{SN2}$

- ▶ **Nonsupported efficient solutions:**

$z(\hat{x})$ is **in interior** of

$\text{conv } Y + \mathbb{R}_{\geq}^p \rightarrow Y_{NN}$



Classification of Efficient Solutions (3/3)

Summary

- ▶ Y_N : the set of nondominated points
 - ▶ Y_{SN} : the set of supported points
 - ▶ Y_{SN1} : the set of extreme supported points
 - ▶ Y_{SN2} : the set of non-extreme supported points
 - ▶ Y_{NN} : the set of nonsupported points
 - ▶ $X_E, z(X_E) = Y_N$: a complete set of efficient solutions
 - ▶ $X_{SE}, z(X_{SE}) = Y_{SN}$: a complete set of supported solutions
 - ▶ $X_{SE1}, z(X_{SE1}) = Y_{SN1}$: a complete set of extreme supported solutions
 - ▶ $X_{SE2}, z(X_{SE2}) = Y_{SN2}$: a complete set of non-extreme supported solutions
 - ▶ $X_{NE}, z(X_{NE}) = Y_{NN}$: a complete set of nonsupported solutions
- with minimal and maximal case

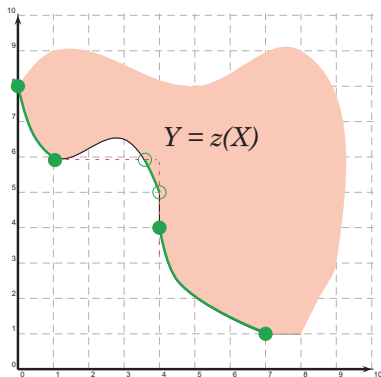
Remarkable points

Remarkable points

Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$

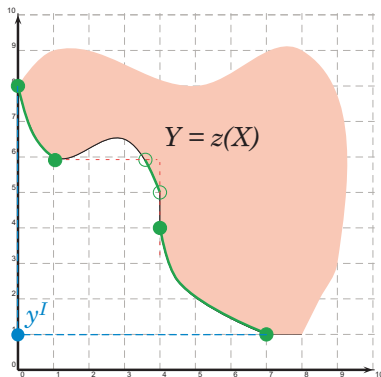


Remarkable points

Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$

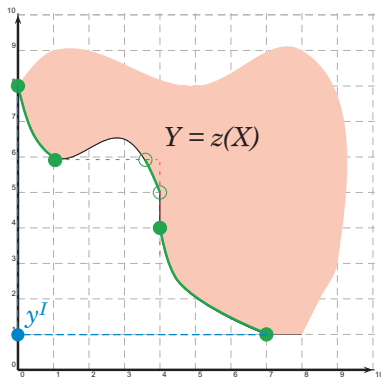


Remarkable points

Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$

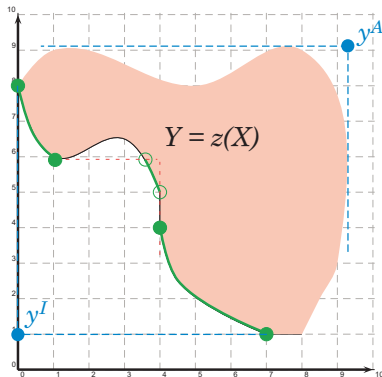


Remarkable points

Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$

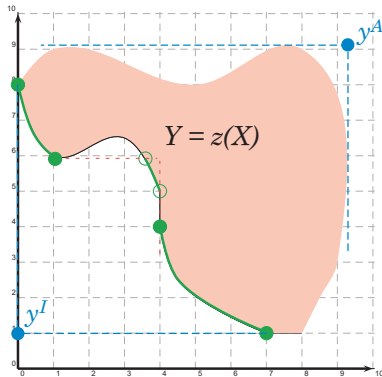


Remarkable points

Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$



Remarkable points

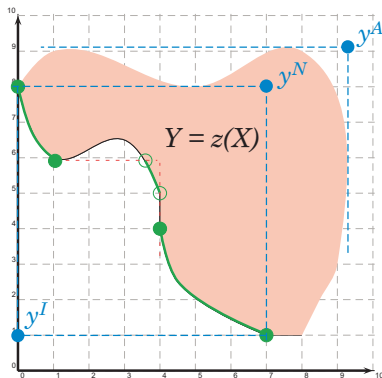
Ideal, anti-ideal and nadir points

Given a MOP:

- ▶ Ideal Point y^I
 $y_k^I = \min\{y_k : y \in Y\}$
- ▶ Anti-Ideal Point y^A
 $y_k^A = \max\{y_k : y \in Y\}$
- ▶ Nadir Point y^N
 $y_k^N = \max\{y_k : y \in Y_N\}$

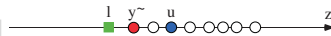
Assumptions:

- ▶ $X_E \neq \emptyset$
- ▶ $y^I \neq y^N$ (the objective functions have not a common optimal solution)



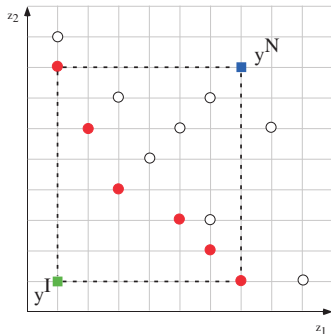
Bound and Bound sets

- ▶ Single-objective case:
(Single) lower and upper bounds
 l and u on the (single) optimal value $\tilde{y}$
- ▶ Multi-objective case:
(Single) bounds on the whole set Y_N
naturally defined by the ideal point y^I and
the nadir point y^N
- ▶ y^I and y^N generally located “far away”
from Y_N
⇒ Need to use several points to bound!



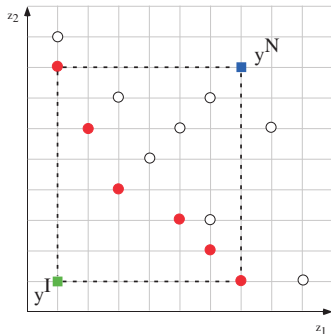
Bound and Bound sets

- ▶ Single-objective case:
(**Single**) lower and upper bounds l and u on the (**single**) optimal value $\tilde{y}$
- ▶ Multi-objective case:
(**Single**) bounds on the whole **set** Y_N naturally defined by the ideal point y^I and the nadir point y^N
- ▶ y^I and y^N generally located “far away” from Y_N
⇒ Need to use several points to bound!



Bound and Bound sets

- ▶ Single-objective case:
(**Single**) lower and upper bounds l and u on the (**single**) optimal value $\tilde{y}$
- ▶ Multi-objective case:
(**Single**) bounds on the whole **set** Y_N naturally defined by the ideal point y^I and the nadir point y^N
- ▶ y^I and y^N generally located “far away” from Y_N
 $\Rightarrow$ Need to use several points to bound!



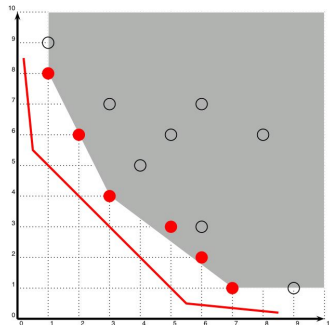
Bound sets: Definition (Ehrgott and Gandibleux, 2007)

1. Lower bound set L (for Y_N)

- ▶ is $\mathbb{R}^p_{\leq}$ -closed
- ▶ is $\mathbb{R}^p_{\leq}$ -bounded
- ▶ $Y_N \subset L + \mathbb{R}^p_{\geq}$
- ▶ $L \subset (L + \mathbb{R}^p_{\geq})_N$

2. Upper bound set U (for Y_N)

- ▶ is $\mathbb{R}^p_{\leq}$ -closed
- ▶ is $\mathbb{R}^p_{\leq}$ -bounded
- ▶ $Y_N \subset \text{cl}[(U + \mathbb{R}^p_{\geq})^c]$
- ▶ $U \subset (U + \mathbb{R}^p_{\geq})_N$



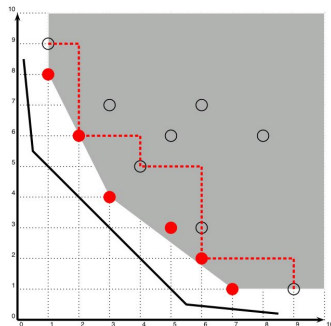
Bound sets: Definition (Ehrgott and Gandibleux, 2007)

1. Lower bound set L (for Y_N)

- ▶ is $\mathbb{R}_{\geq}^p$ -closed
- ▶ is $\mathbb{R}_{\geq}^p$ -bounded
- ▶ $Y_N \subset L + \mathbb{R}_{\geq}^p$
- ▶ $L \subset (L + \mathbb{R}_{\geq}^p)_N$

2. Upper bound set U (for Y_N)

- ▶ is $\mathbb{R}_{\leq}^p$ -closed
- ▶ is $\mathbb{R}_{\leq}^p$ -bounded
- ▶ $Y_N \subset \text{cl}[(U + \mathbb{R}_{\leq}^p)^c]$
- ▶ $U \subset (U + \mathbb{R}_{\leq}^p)_N$



Summary (in the linear case)

Multiobjective Optimization Problem (MOP) notations

$$\begin{array}{ll}\min & F(x) \\ \text{s.t.} & x \in X\end{array}$$

where:

$$\begin{array}{ll}X \subseteq \mathbb{R}^n & \longrightarrow \text{the set of feasible solutions} \\ Y = F(X) \subseteq \mathbb{R}^p & \longrightarrow \text{the set of images}\end{array}$$

Computing sets X_E and Y_N for MOP:

$$y = F(x)$$

$y^* \in Y$ is nondominated, if $\nexists y \in Y$ such that $y_i \leq y_i^*, \forall i$ and $y \neq y^*$.

Y_N is the set of nondominated points.

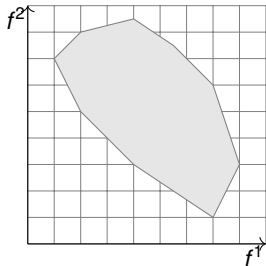
$x^* \in X$ is efficient if y^* is nondominated.

X_E is a complete set of efficient solutions.

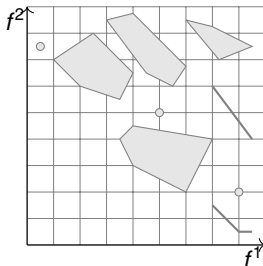
Examples of Y and Y_N when $p = 2$

Computing Y_N for Multiobjective (linear) Optimization Problems (MOP):

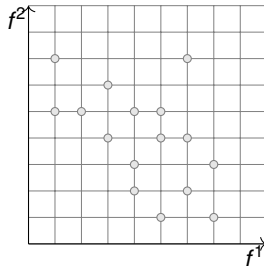
$$x \in \mathbb{R}^{n_1}$$



$$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$$



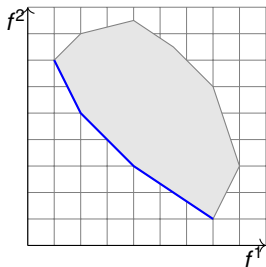
$$x \in \mathbb{Z}^{n_2}$$



Examples of Y and Y_N when $p = 2$

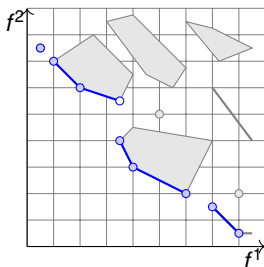
Computing Y_N for Multiobjective (linear) Optimization Problems (MOP):

$$x \in \mathbb{R}^{n_1}$$



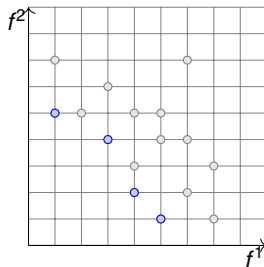
Continuous
nondominated set
(edges)

$$x \in \mathbb{R}^{n_1} \times \mathbb{Z}^{n_2}$$



Nondominated set
composed of edges that
are either closed,
half-open, open or
reduced to a point

$$x \in \mathbb{Z}^{n_2}$$



Discrete set of
nondominated
points

Exercise

Exercise

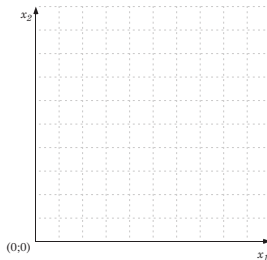
For the given MOP, determine:

- ▶ X_E
- ▶ Y_N

Exercise

MOP1:

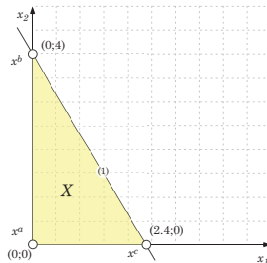
$$\begin{array}{ll}\max z^1(x) &= x_1 + x_2 \\ \max z^2(x) &= x_1 \\ \text{subject to} & 5x_1 + 3x_2 \leq 12 \\ & x_1, x_2 \geq 0\end{array}$$



Exercise

MOP1:

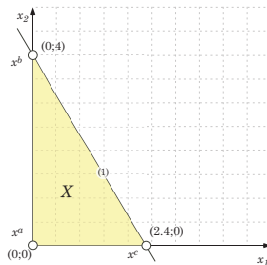
$$\begin{aligned} \max z^1(x) &= x_1 + x_2 \\ \max z^2(x) &= x_1 \\ \text{subject to} \quad &5x_1 + 3x_2 \leq 12 \quad (1) \\ &x_1, x_2 \geq 0 \end{aligned}$$



Exercise

MOP1: $z(x)$, performances of $x \in X$, and Y , the outcome set?

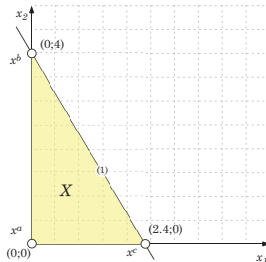
$$\begin{aligned} \max z^1(x) &= x_1 + x_2 \\ \max z^2(x) &= x_1 \\ \text{subject to} \quad &5x_1 + 3x_2 \leq 12 \quad (1) \\ &x_1, x_2 \geq 0 \end{aligned}$$



Exercise

MOP1:

$$\begin{aligned}\max z^1(x) &= x_1 + x_2 \\ \max z^2(x) &= x_1 \\ \text{subject to} \quad &5x_1 + 3x_2 \leq 12 \quad (1) \\ &x_1, x_2 \geq 0\end{aligned}$$



$$x^a = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \rightarrow y(x^a) = \begin{pmatrix} z^1(x^a) \\ z^2(x^a) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

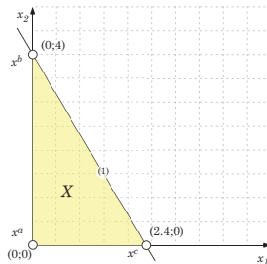
$$x^b = \begin{pmatrix} 0 \\ 4 \end{pmatrix} \rightarrow y(x^b) = \begin{pmatrix} z^1(x^b) \\ z^2(x^b) \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \end{pmatrix}$$

$$x^c = \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} \rightarrow y(x^c) = \begin{pmatrix} z^1(x^c) \\ z^2(x^c) \end{pmatrix} = \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix}$$

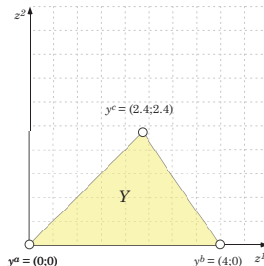
Exercise

MOP1:

$$\begin{aligned}\max z^1(x) &= x_1 + x_2 \\ \max z^2(x) &= x_1 \\ \text{subject to} & \quad 5x_1 + 3x_2 \leq 12 \quad (1) \\ & \quad x_1, x_2 \geq 0\end{aligned}$$



$$\begin{aligned}x^a &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} & y(x^a) &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ x^b &= \begin{pmatrix} 0 \\ 4 \end{pmatrix} & y(x^b) &= \begin{pmatrix} 4 \\ 0 \end{pmatrix} \\ x^c &= \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} & y(x^c) &= \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix}\end{aligned}$$



Exercise

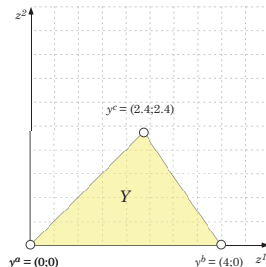
MOP1: Values of Y_N , Y_{SN} , and Y_{NN} ?

► $Y_N = ?$

► $Y_{SN} = ?$

► $Y_{NN} = ?$

$$\begin{aligned} x^a &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} & y(x^a) &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ x^b &= \begin{pmatrix} 0 \\ 4 \end{pmatrix} & y(x^b) &= \begin{pmatrix} 4 \\ 0 \end{pmatrix} \\ x^c &= \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} & y(x^c) &= \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix} \end{aligned}$$



Exercise

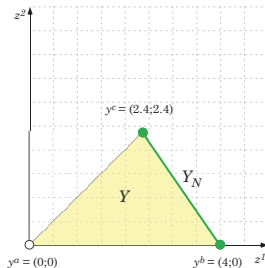
MOP1:

► $Y_N = \{y \in [y^b, y^c]\}$

► $Y_{SN} = Y_N$

► $Y_{NN} = \emptyset$

$$\begin{aligned} x^a &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} & y(x^a) &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ x^b &= \begin{pmatrix} 0 \\ 4 \end{pmatrix} & y(x^b) &= \begin{pmatrix} 4 \\ 0 \end{pmatrix} \\ x^c &= \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} & y(x^c) &= \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix} \end{aligned}$$



Exercise

MOP1:

Remarkable points on Y ?

► $Y_N = \{y \in [y^b, y^c]\}$

► $y^I = ?$

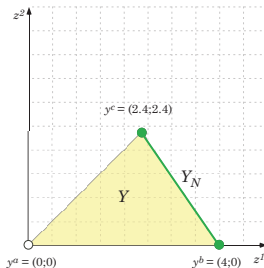
► $Y_{SN} = Y_N$

► $y^A = ?$

► $Y_{NN} = \emptyset$

► $y^N = ?$

$$\begin{aligned} x^a &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} & y(x^a) &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ x^b &= \begin{pmatrix} 0 \\ 4 \end{pmatrix} & y(x^b) &= \begin{pmatrix} 4 \\ 0 \end{pmatrix} \\ x^c &= \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} & y(x^c) &= \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix} \end{aligned}$$



Exercise

MOP1:

$$\triangleright Y_N = \{y \in [y^b, y^c]\}$$

$$\triangleright Y_{SN} = Y_N$$

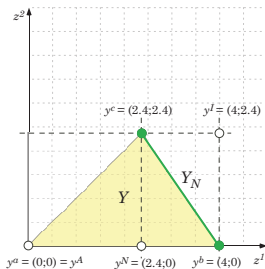
$$\triangleright Y_{NN} = \emptyset$$

$$\triangleright y^I = \begin{pmatrix} 4 \\ 12/5 \end{pmatrix}$$

$$\triangleright y^A = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\triangleright y^N = \begin{pmatrix} 12/5 \\ 0 \end{pmatrix}$$

$$\begin{aligned} x^a &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} & y(x^a) &= \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ x^b &= \begin{pmatrix} 0 \\ 4 \end{pmatrix} & y(x^b) &= \begin{pmatrix} 4 \\ 0 \end{pmatrix} \\ x^c &= \begin{pmatrix} 12/5 \\ 0 \end{pmatrix} & y(x^c) &= \begin{pmatrix} 12/5 \\ 12/5 \end{pmatrix} \end{aligned}$$



Exercise

MOP1: discussion

- ▶ x : continuous and non negative variables
- ▶ X : linear constraints
- ▶ f : linear objectives
- ▶ Y : convex, defined by a polyedron

⇒ Class of MOLP

- ▶ “easy” to solve
- ▶ $Y_{SN} = Y_N$ ($Y_{NN} = \emptyset$)
- ▶ Algorithms:
 - ▶ Parametric method ($p = 2$), MO simplex method ($p > 2$)
 - ▶ Interior point method
 - ▶ Benson method
- ▶ Solver:
 - ▶ BENSOLVE (Löhne, 2014) <http://www.bensolve.org/>

Exercise

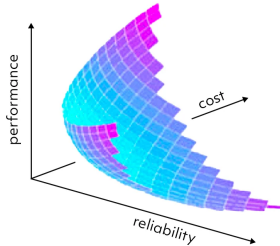
MOP1: discussion

- ▶ x : continuous and non negative variables
- ▶ X : linear constraints
- ▶ f : linear objectives
- ▶ Y : convex, defined by a polyedron

⇒ Class of MOLP

- ▶ “easy” to solve
- ▶ $Y_{SN} = Y_N$ ($Y_{NN} = \emptyset$)
- ▶ Algorithms:
 - ▶ Parametric method ($p = 2$), MO simplex method ($p > 2$)
 - ▶ Interior point method
 - ▶ Benson method
- ▶ Solver:
 - ▶ BENSOLVE (Löhne, 2014) <http://www.bensolve.org/>

Overview of Algorithms



General Methods and Algorithms for solving a MOP

Methods and Algorithms for solving a MOP

Exact vs approximation methods

Computing the efficient solutions of a MOP:

- ▶ **Exact methods:**

- Scalarizing Methods

- weighted sum, ϵ -constraint, compromise programming...

- Non scalarizing Methods

- labeling, dynamic programming, branch and bound, two phases...

- ▶ **Approximation methods:**

- With guarantee:

- polynomial time approximation algorithms, (F)PTAS

- Without guarantee:

- $\hat{A}$ (meta)heuristics, MO(M)H

- ▶ **Hybrid methods:**

- Approximation $\Leftrightarrow$ exact methods

Methods and Algorithms for solving a MOP

Exact vs approximation methods

Computing the efficient solutions of a MOP:

- ▶ **Exact methods:**
 - Scalarizing Methods
weighted sum, ϵ -constraint, compromise programming...
 - Non scalarizing Methods
labeling, dynamic programming, branch and bound, two phases...
- ▶ **Approximation methods:**
 - With guarantee:
polynomial time approximation algorithms, (F)PTAS
 - Without guarantee:
 $\hat{A}$ (meta)heuristics, MO(M)H
- ▶ **Hybrid methods:**
 - Approximation $\Leftrightarrow$ exact methods

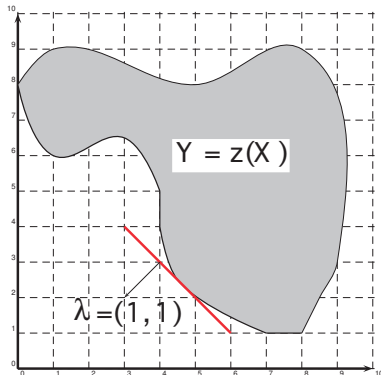
Scalarizing Methods

Weighted Sum Method

The Weighted Sum Method

- ▶ A natural idea : Aggregate objectives
- ▶ Let $\lambda \in \mathbb{R}_{\geq 0}^p$ be a weight vector
A weighted sum problem is defined by

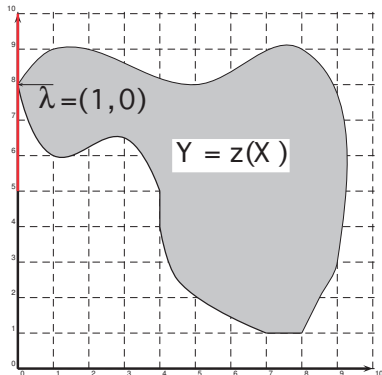
$$\min \left\{ \sum_{k=1}^p \lambda_k z_k(x) : x \in X \right\} \quad (1)$$



The Weighted Sum Method

- ▶ A natural idea : Aggregate objectives
- ▶ Let $\lambda \in \mathbb{R}_{\geq 0}^p$ be a weight vector
A weighted sum problem is defined by

$$\min \left\{ \sum_{k=1}^p \lambda_k z_k(x) : x \in X \right\} \quad (1)$$



The Weighted Sum Method

- ▶ A natural idea : Aggregate objectives
- ▶ Let $\lambda \in \mathbb{R}_{\geq}^p$ be a weight vector
A weighted sum problem is defined by

$$\min \left\{ \sum_{k=1}^p \lambda_k z_k(x) : x \in X \right\} \quad (1)$$

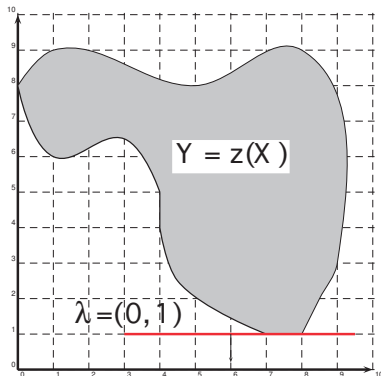
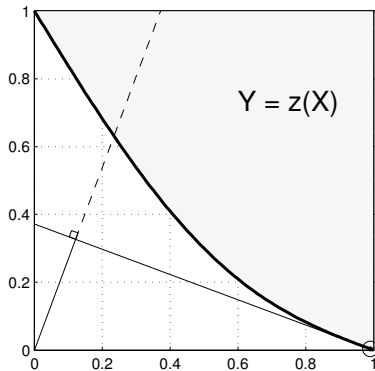


Illustration with Two objectives

Y is convex



Y is not convex

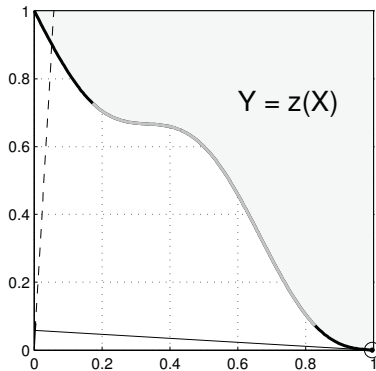
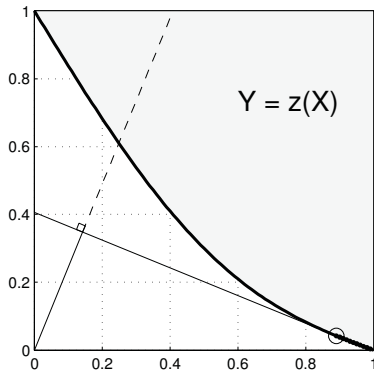


Illustration with Two objectives

Y is convex



Y is not convex

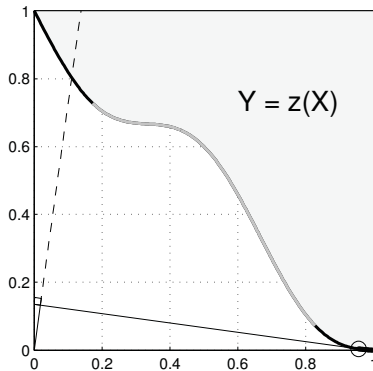
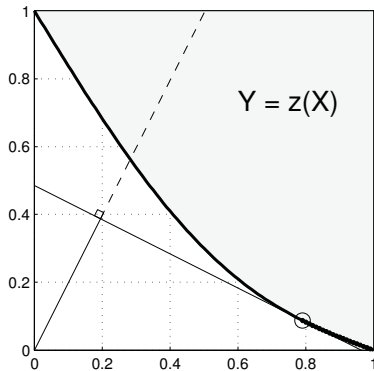


Illustration with Two objectives

Y is convex



Y is not convex

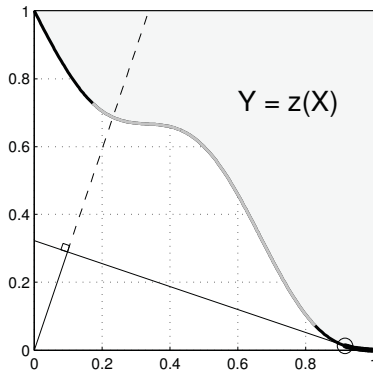
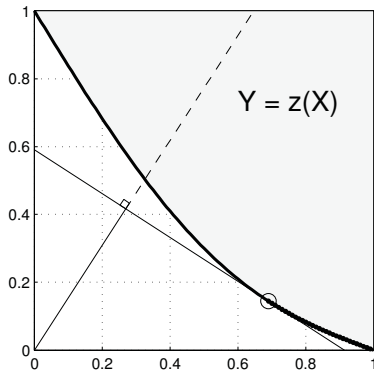


Illustration with Two objectives

Y is convex



Y is not convex

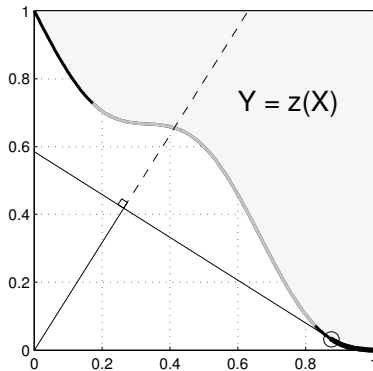
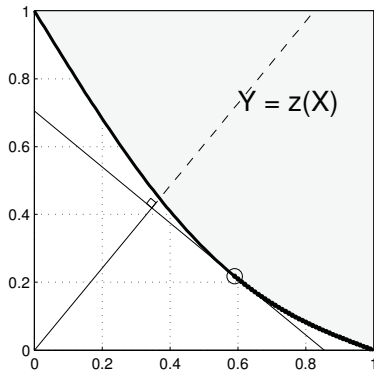


Illustration with Two objectives

Y is convex



Y is not convex

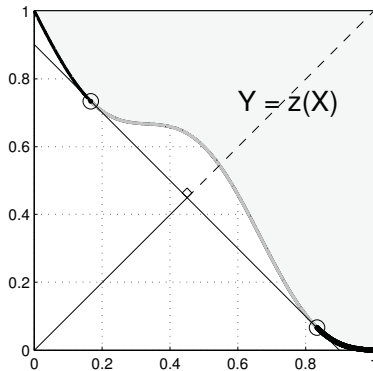
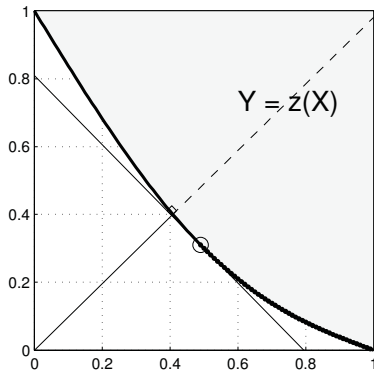


Illustration with Two objectives

Y is convex



Y is not convex

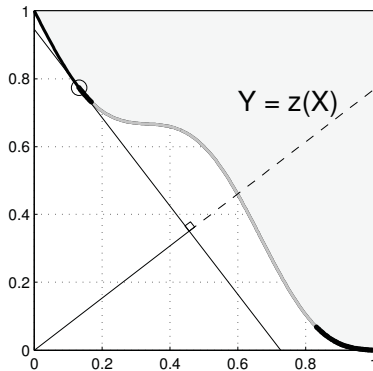
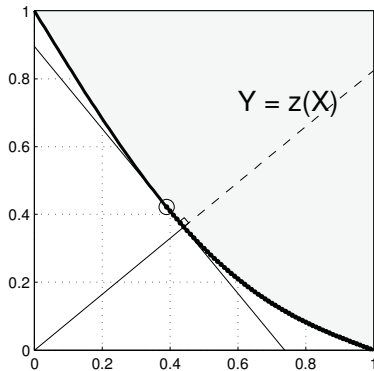


Illustration with Two objectives

Y is convex



Y is not convex

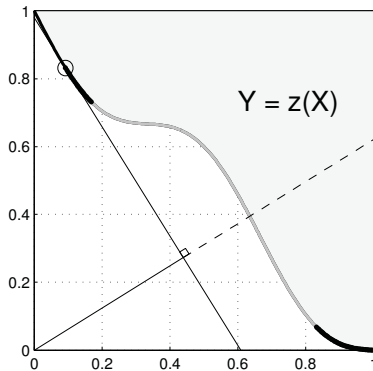
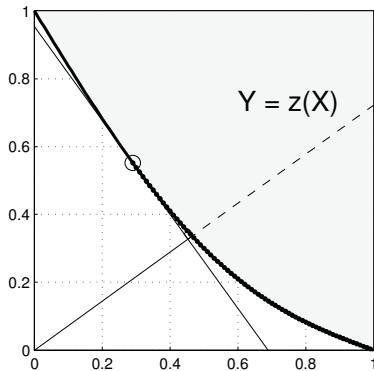


Illustration with Two objectives

Y is convex



Y is not convex

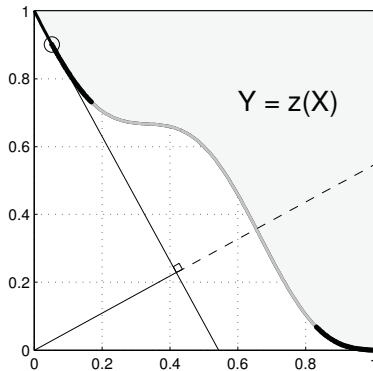
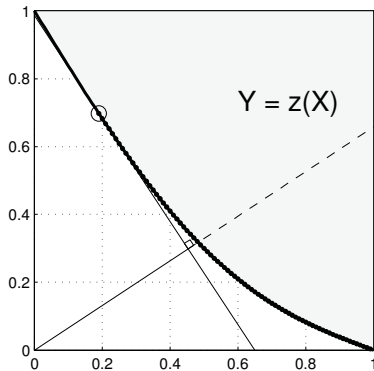


Illustration with Two objectives

Y is convex



Y is not convex

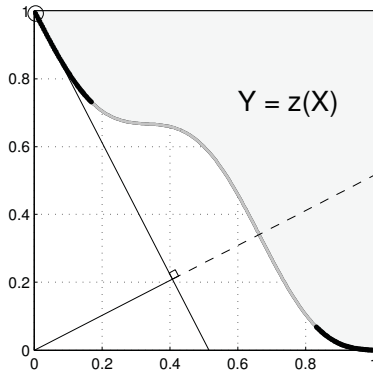
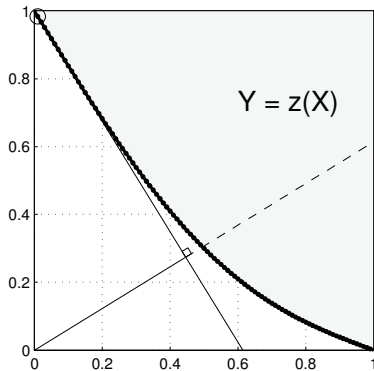
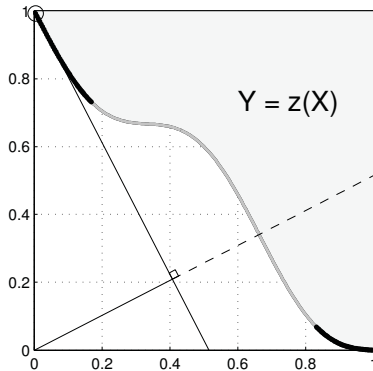


Illustration with Two objectives

Y is convex



Y is not convex



Algorithm of Aneja and Nair (Dichotomic Algorithm)

Algorithm (Aneja and Nair 1979)

- ▶ Compute $x^{(1,2)}$ and $x^{(2,1)}$ two lexicographic optimal solutions for resp. $z^{(1,2)}$ and $z^{(2,1)}$
- ▶ $\tilde{X} \leftarrow \{x^{(1,2)}, x^{(2,1)}\}$
- ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^{(1,2)}, x^{(2,1)}, \tilde{X})$

Algorithm (solveRecursion)

Input : $x^1, x^2 \in X_{SE}$, $\tilde{X} \subseteq X_{SE}$

- ▶ $\lambda_1 \leftarrow z_2(x^1) - z_2(x^2); \lambda_2 \leftarrow z_1(x^2) - z_1(x^1)$
- ▶ $x \leftarrow \text{solveWeightedSum}(\lambda); \tilde{X} \leftarrow \tilde{X} \cup \{x\}$
- ▶ If $(\lambda_1 z_1(x) + \lambda_2 z_2(x) < \lambda_1 z_1(x^1) + \lambda_2 z_2(x^1))$ Then
 - ▶ $\tilde{X} \leftarrow \text{solveRecursion}(x^1, x, \tilde{X})$
 - ▶ $\tilde{X} \leftarrow \text{solveRecursion}(x^2, x, \tilde{X})$

Algorithm (Aneja and Nair 1979)

- ▶ Compute $x^{(1,2)}$ and $x^{(2,1)}$ two lexicographic optimal solutions for resp. $z^{(1,2)}$ and $z^{(2,1)}$
- ▶ $\tilde{X} \leftarrow \{x^{(1,2)}, x^{(2,1)}\}$
- ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^{(1,2)}, x^{(2,1)}, \tilde{X})$

Algorithm (solveRecursion)

Input : $x^1, x^2 \in X_{SE}$, $\tilde{X} \subseteq X_{SE}$

- ▶ $\lambda_1 \leftarrow z_2(x^1) - z_2(x^2); \lambda_2 \leftarrow z_1(x^2) - z_1(x^1)$
- ▶ $x \leftarrow \text{solveWeightedSum}(\lambda); \tilde{X} \leftarrow \tilde{X} \cup \{x\}$
- ▶ If $(\lambda_1 z_1(x) + \lambda_2 z_2(x) < \lambda_1 z_1(x^1) + \lambda_2 z_2(x^1))$ Then
 $\lambda_1 \leftarrow \lambda_1 + \lambda_2 z_1(x)$
 $\lambda_2 \leftarrow \lambda_2 - \lambda_1 z_2(x)$

Algorithm (Aneja and Nair 1979)

- ▶ Compute $x^{(1,2)}$ and $x^{(2,1)}$ two lexicographic optimal solutions for resp. $z^{(1,2)}$ and $z^{(2,1)}$
- ▶ $\tilde{X} \leftarrow \{x^{(1,2)}, x^{(2,1)}\}$
- ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^{(1,2)}, x^{(2,1)}, \tilde{X})$

Algorithm (solveRecursion)

Input : $x^1, x^2 \in X_{SE}, \tilde{X} \subseteq X_{SE}$

- ▶ $\lambda_1 \leftarrow z_2(x^1) - z_2(x^2); \lambda_2 \leftarrow z_1(x^2) - z_1(x^1)$
- ▶ $x \leftarrow \text{solveWeightedSum}(\lambda); \tilde{X} \leftarrow \tilde{X} \cup \{x\}$
- ▶ If $(\lambda_1 z_1(x) + \lambda_2 z_2(x) < \lambda_1 z_1(x^1) + \lambda_2 z_2(x^1))$ Then
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^1, x, \tilde{X})$
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x, x^2, \tilde{X})$

Algorithm (Aneja and Nair 1979)

- ▶ Compute $x^{(1,2)}$ and $x^{(2,1)}$ two lexicographic optimal solutions for resp. $z^{(1,2)}$ and $z^{(2,1)}$
- ▶ $\tilde{X} \leftarrow \{x^{(1,2)}, x^{(2,1)}\}$
- ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^{(1,2)}, x^{(2,1)}, \tilde{X})$

Algorithm (solveRecursion)

Input : $x^1, x^2 \in X_{SE}, \tilde{X} \subseteq X_{SE}$

- ▶ $\lambda_1 \leftarrow z_2(x^1) - z_2(x^2); \lambda_2 \leftarrow z_1(x^2) - z_1(x^1)$
- ▶ $x \leftarrow \text{solveWeightedSum}(\lambda); \tilde{X} \leftarrow \tilde{X} \cup \{x\}$
- ▶ If $(\lambda_1 z_1(x) + \lambda_2 z_2(x) < \lambda_1 z_1(x^1) + \lambda_2 z_2(x^1))$ Then
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^1, x, \tilde{X})$
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x, x^2, \tilde{X})$

Algorithm (Aneja and Nair 1979)

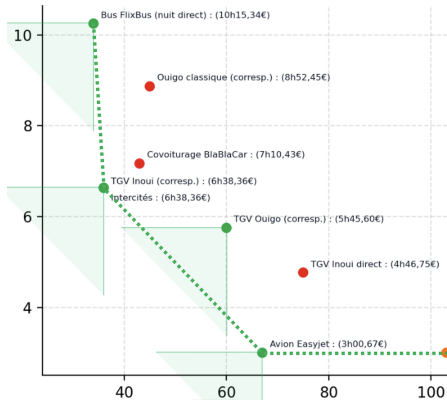
- ▶ Compute $x^{(1,2)}$ and $x^{(2,1)}$ two lexicographic optimal solutions for resp. $z^{(1,2)}$ and $z^{(2,1)}$
- ▶ $\tilde{X} \leftarrow \{x^{(1,2)}, x^{(2,1)}\}$
- ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^{(1,2)}, x^{(2,1)}, \tilde{X})$

Algorithm (solveRecursion)

Input : $x^1, x^2 \in X_{SE}, \tilde{X} \subseteq X_{SE}$

- ▶ $\lambda_1 \leftarrow z_2(x^1) - z_2(x^2); \lambda_2 \leftarrow z_1(x^2) - z_1(x^1)$
- ▶ $x \leftarrow \text{solveWeightedSum}(\lambda); \tilde{X} \leftarrow \tilde{X} \cup \{x\}$
- ▶ If $(\lambda_1 z_1(x) + \lambda_2 z_2(x) < \lambda_1 z_1(x^1) + \lambda_2 z_2(x^1))$ Then
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x^1, x, \tilde{X})$
 - ▶ $\tilde{X} \leftarrow \text{SolveRecursion}(x, x^2, \tilde{X})$

Supported and Non-Supported Efficient Solutions



Scalarizing Methods

The ε -constraint Method

The ε -constraint Method

- ▶ Given an instance of a MOCO/MOIP problem, a complete set can be computed using ε -constraint method
- ▶ All efficient solution $\bar{x}$ is an optimal solution of a problem

$$\min_{x \in X} \{z_l(x) : z_k(x) \leq \varepsilon_k, k \neq l\} \quad (2)$$

Algorithm (ε -constraint with adaptive step)

1. Initialization:

- ▶ Determine x^1 a lexicographic optimal solution for $z^{(1,2)}$
- ▶ $\tilde{X} \leftarrow \{x^1\}$
- ▶ $\varepsilon_1 \leftarrow z_2(x^1) - \epsilon$

2. While problem (2) is feasible do

- ▶ Find $\tilde{x}$ an optimal solution of (2)
- ▶ $\tilde{X} \leftarrow \tilde{X} \cup \{\tilde{x}\}$
- ▶ $\varepsilon_1 \leftarrow z_2(\tilde{x}) - \epsilon$

3. Filter dominated solutions in $\tilde{X}$

Output: $\tilde{X}$ contains one solution for each nondominated point, i.e. a minimal complete set X_{E_m}

The ε -constraint Method

- ▶ Given an instance of a MOCO/MOIP problem, a complete set can be computed using ε -constraint method
- ▶ All efficient solution $\bar{x}$ is an optimal solution of a problem

$$\min_{x \in X} \{z_l(x) : z_k(x) \leq \varepsilon_k, k \neq l\} \quad (2)$$

Algorithm (ε -constraint with adaptive step)

1. Initialization:

- ▶ Determine x^1 a lexicographic optimal solution for $z^{(1,2)}$
- ▶ $\tilde{X} \leftarrow \{x^1\}$
- ▶ $\varepsilon_1 \leftarrow z_2(x^1) - \epsilon$

2. While problem (2) is feasible do

- ▶ Let $\hat{x}$ be an optimal solution of (2)
- ▶ $\tilde{X} \leftarrow \tilde{X} \cup \{\hat{x}\}$
- ▶ $\varepsilon_1 \leftarrow z_2(\hat{x}) - \epsilon$

3. Filter dominated solutions in $\tilde{X}$

Output: $\tilde{X}$ contains one solution for each nondominated point, i.e. a minimal complete set X_{Em}

The ε -constraint Method

- ▶ Given an instance of a MOCO/MOIP problem, a complete set can be computed using ε -constraint method
- ▶ All efficient solution $\bar{x}$ is an optimal solution of a problem

$$\min_{x \in X} \{z_l(x) : z_k(x) \leq \varepsilon_k, k \neq l\} \quad (2)$$

Algorithm (ε -constraint with adaptive step)

1. Initialization:

- ▶ Determine x^1 a lexicographic optimal solution for $z^{(1,2)}$
- ▶ $\tilde{X} \leftarrow \{x^1\}$
- ▶ $\varepsilon_1 \leftarrow z_2(x^1) - \epsilon$

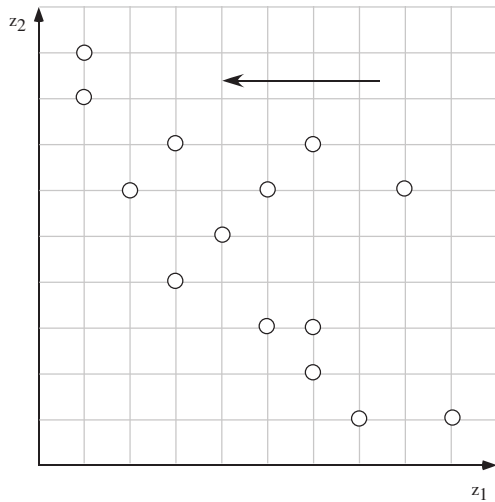
2. While problem (2) is feasible do

- ▶ Let $\hat{x}$ be an optimal solution of (2)
- ▶ $\tilde{X} \leftarrow \tilde{X} \cup \{\hat{x}\}$
- ▶ $\varepsilon_1 \leftarrow z_2(\hat{x}) - \epsilon$

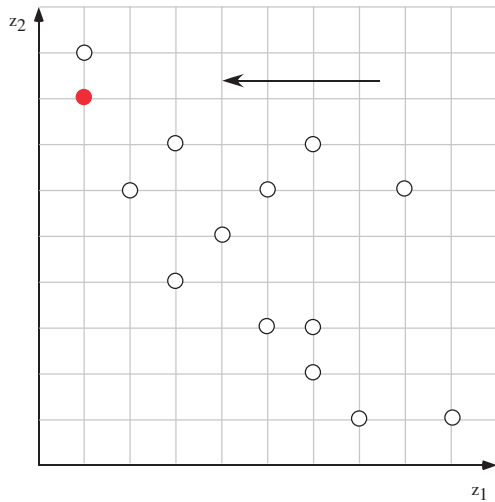
3. Filter dominated solutions in $\tilde{X}$

Output: $\tilde{X}$ contains one solution for each nondominated point, i.e. a minimal complete set X_{Em}

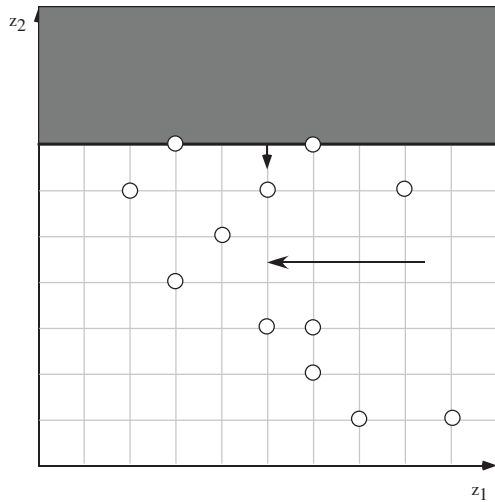
Illustration



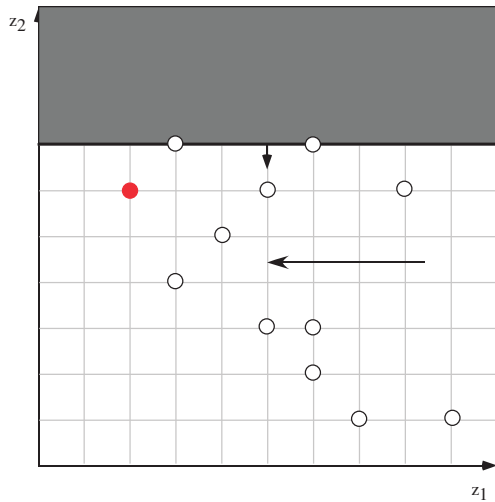
Illustration



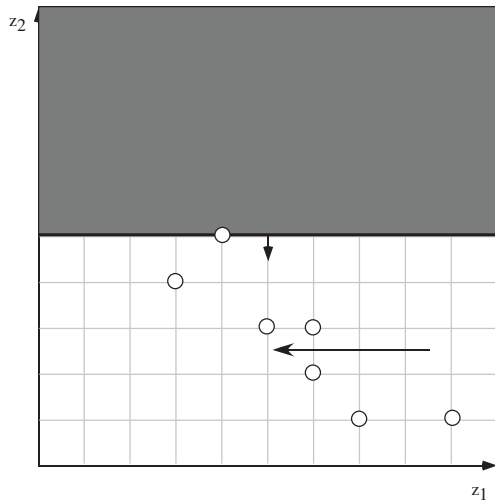
Illustration



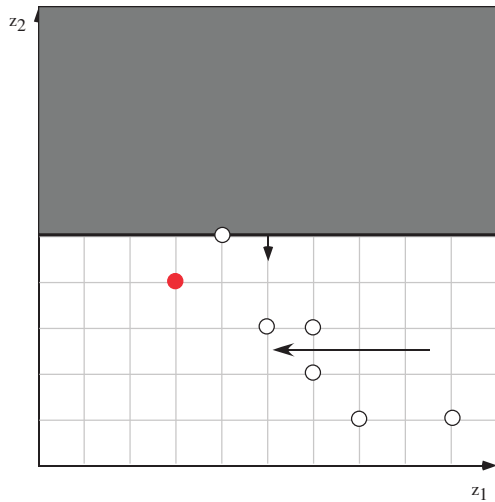
Illustration



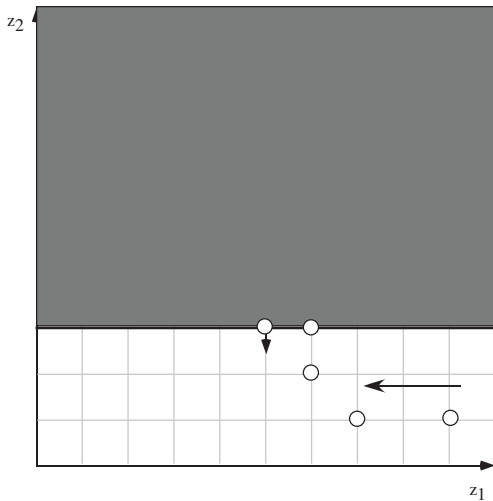
Illustration



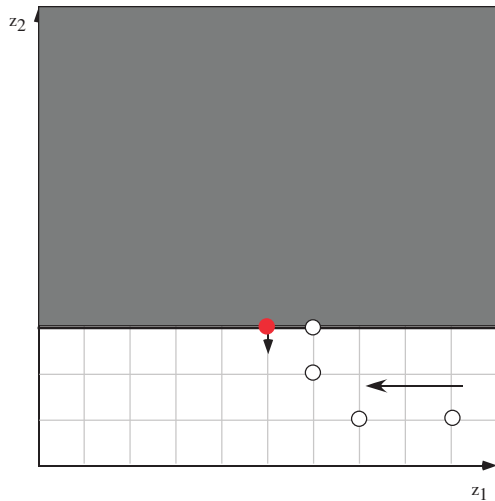
Illustration



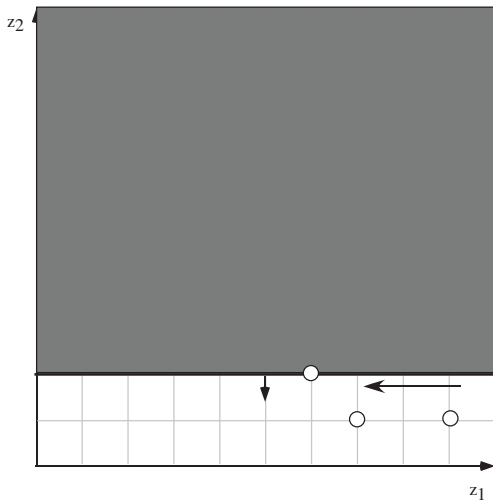
Illustration



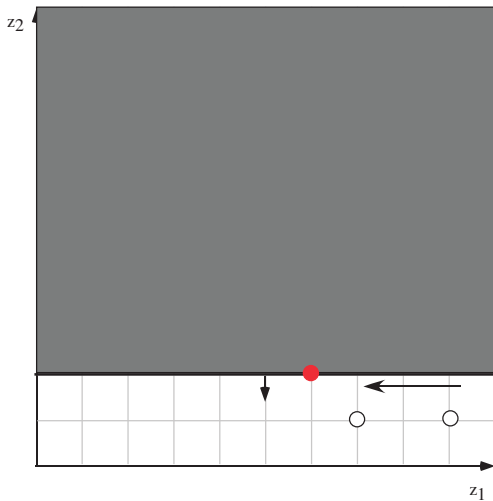
Illustration



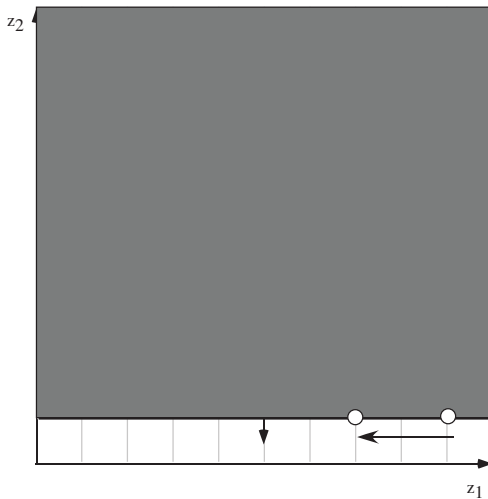
Illustration



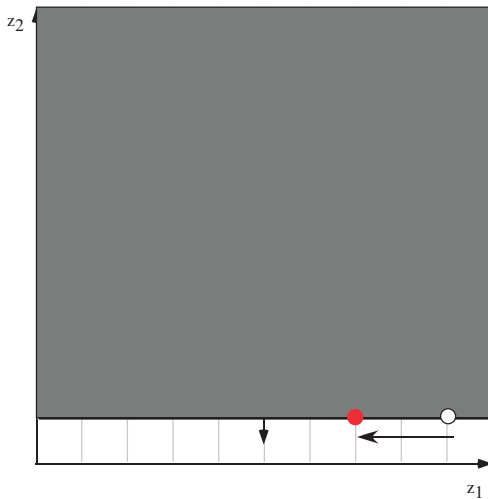
Illustration



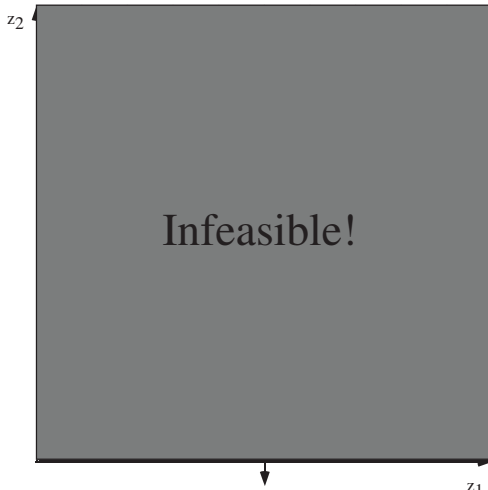
Illustration



Illustration



Illustration



Multiobjective case: ε -constraint (like) with adaptive step

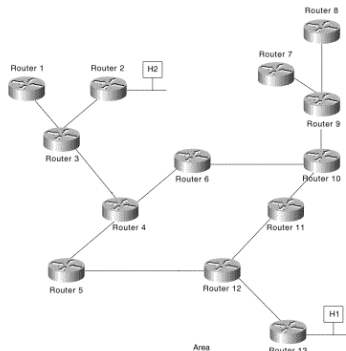
- ▶ M. Laumanns, L. Thiele, E. Zitzler. An adaptive scheme to generate the Pareto front based on the ε -constraint method. In: J. Branke, K. Deb, K. Miettinen, R.E. Steuer (eds.) Practical Approaches to Multi-Objective Optimization, number 04461 in Dagstuhl Seminar Proceedings, Dagstuhl, Germany, 2005. Internationales Begegnungs und Forschungszentrum für Informatik (IBFI), Schloss Dagstuhl, Germany.
- ▶ M. Ozlen, M. Azizoglu. Multi-objective integer programming: a general approach for generating all non-dominated solutions. Eur. J. Oper. Res. 199, 25-35 (2009)
- ▶ M. Ozlen, B.A. Burton, C.A.G MacRae. Multi-objective integer programming: an improved recursive algorithm. J. Optim. Theory and Appl. 160(2), 470-482 (2014)
- ▶ B. Lokman, M. Köksalan. Finding all nondominated points of multi-objective integer programs. J. Global Optim. 57, 347-365 (2013)
- ▶ G. Kirlik, S. Sayin. A new algorithm for generating all nondominated solutions of multiobjective discrete optimization problems. Eur. J. Oper. Res. 232, 479-488 (2014)
- ▶ K. Dächert, K. Klamroth. A linear bound on the number of scalarizations needed to solve tricriteria optimization problems. Journal of Global Optimization, Volume 61, Issue 4, pp 643–676. 2015.
- ▶ S. Tamby, D. Vanderpooten. Enumeration of the Nondominated Set of Multiobjective Discrete Optimization Problems. INFORMS Journal on Computing 33(1):72-85, 2020
- ▶ M. Ángel Domínguez-Ríos, Fr. Chicano, E. Alba, Effective anytime algorithm for multiobjective combinatorial optimization problems, Information Sciences, Volume 565, Pages 210-228, 2021.

Non scalarizing Methods

A Labeling Algorithm for Multi-Objective Shortest Paths

Application
Multiobjective Routing in IP Networks
Research collaboration project with ALCATEL

Routing information at IP level



Current solution following the OSPF protocol (Open Shortest Path First) :

compute one shortest path in real time using **the Dijkstra algorithm** where **the number of hops** is minimized

Other criteria need to be considered :

- **delays**
- **available bandwidth**
- **costs**
- **reliability**, etc.

⇒ Design a **multicriteria and multiconstrained IP routing** compatible with a real-time environment

From Dijkstra
to
Hansen and Martins
for
multiobjective shortest paths

Algorithms of Label setting

Algorithm	P	Type	Costs	Results
Dijkstra	1	$\sum$	≥ 0	$1 \times n$
Dijkstra bis	1	$\sum$	$= 1$	$1 \times n$
Hansen	2	$\sum + \text{max-min}$	≥ 0	1×1 : minimal set
Martins	≥ 1	$P \sum$	≥ 0	$1 \times n$: every eff. sol.
	≥ 2	$P \sum + \text{max-min}$	≥ 0	$1 \times n$: every eff. sol.

Multiobjective Shortest Path Problem

Martins' algorithm generalizes Dijkstra's algorithm to the multiobjective case

- ▶ Assumption: $c_{ij}^k \geq 0$ for all $(i, j) \in A$ and $k \in \{1, \dots, p\}$
- ▶ The main idea justifying Dijkstra's algorithm remains verified

Theorem:

Let P_{st} be an **efficient** path from s to t . Then any subpath P_{uv} from u to v , where u and v are vertices on P_{st} is an **efficient** path from u to v

- ▶ There may exist several efficient paths between any pair of node $\implies$
Multiple Labels are required for each node

Martins' Algorithm

Ingredients of the algorithm:

- ▶ Sets of nondominated labels at each node
- ▶ Store all costs, the predecessor, the label number at the predecessor
- ▶ A list of temporary and permanent labels ($\mathcal{TL}, \mathcal{PL}$)
- ▶ A permanent label defines an efficient path:
Select the lexicographically smallest label from temporary list

Martins' Algorithm

Ingredients of the algorithm:

- ▶ Sets of nondominated labels at each node
- ▶ Store all costs, the predecessor, the label number at the predecessor
- ▶ A list of temporary and permanent labels ($\mathcal{TL}, \mathcal{PL}$)
- ▶ A permanent label defines an efficient path:
Select the lexicographically smallest label from temporary list

Martins' Algorithm

Ingredients of the algorithm:

- ▶ Sets of nondominated labels at each node
- ▶ Store all costs, the predecessor, the label number at the predecessor
- ▶ A list of temporary and permanent labels ($\mathcal{TL}, \mathcal{PL}$)
- ▶ A permanent label defines an efficient path:
Select the lexicographically smallest label from temporary list

Martins' Algorithm

Ingredients of the algorithm:

- ▶ Sets of nondominated labels at each node
- ▶ Store all costs, the predecessor, the label number at the predecessor
- ▶ A list of temporary and permanent labels ($\mathcal{TL}, \mathcal{PL}$)
- ▶ A permanent label defines an efficient path:
Select the lexicographically smallest label from temporary list

Martins' Algorithm

Ingredients of the algorithm:

- ▶ Sets of nondominated labels at each node
- ▶ Store all costs, the predecessor, the label number at the predecessor
- ▶ A list of temporary and permanent labels ($\mathcal{TL}, \mathcal{PL}$)
- ▶ A permanent label defines an efficient path:
Select the lexicographically smallest label from temporary list

Algorithm

Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}; \mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_j, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if it is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Algorithm

Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}; \mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_j, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if it is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Algorithm

Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}; \mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_j, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if it is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Algorithm

Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}$; $\mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_i, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Algorithm

Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}$; $\mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_i, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Algorithm

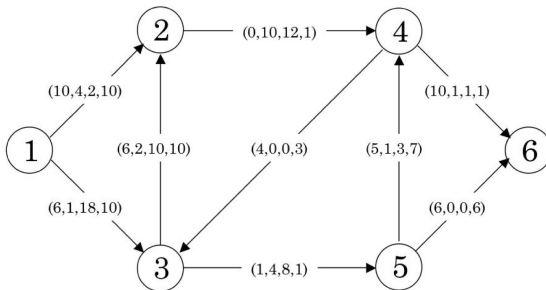
Input: A digraph $G = (V, A, C)$

- ▶ Create Label $L = [0, \dots, 0, \times, \times]$ at node s and let $\mathcal{TL} \leftarrow \{L\}$
- ▶ While $\mathcal{TL} \neq \emptyset$ do
 1. Let label $L = [c^1, \dots, c^p, v_h, l]$ of node v_i be the lexicographically smallest label in $\mathcal{TL}$
 2. $\mathcal{TL} \leftarrow \mathcal{TL} \setminus \{L\}$; $\mathcal{PL} \leftarrow \mathcal{PL} \cup \{L\}$
 3. For all $v_j \in V$ with $(v_i, v_j) \in A$ do
 - ▶ Create Label $L' = [c^1 + c^1(v_i, v_j), \dots, c^p + c^p(v_i, v_j), v_i, k]$ as the next label at node v_j ; $\mathcal{TL} \leftarrow \mathcal{TL} \cup \{L'\}$
 - ▶ Delete all temporary labels of node v_j dominated by L' , delete L' if is dominated by another label of node v_j
- ▶ Extract all efficient paths from s to other nodes of $\mathcal{G}$

Theorem:

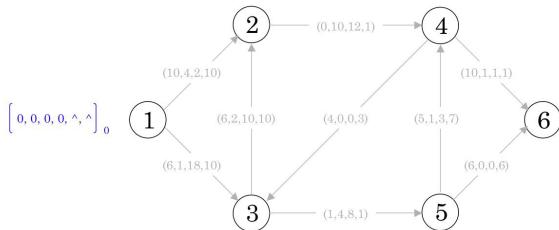
At termination of Martins' algorithm, the permanent labels at each node v_i define the set of efficient paths from node s to node v_i

Example

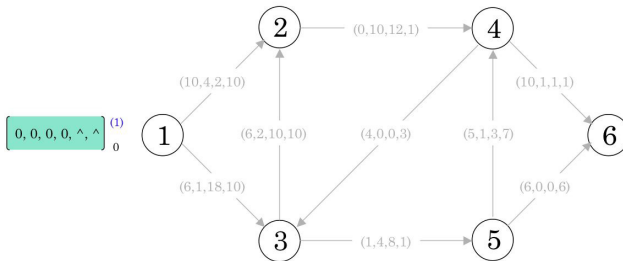


Find all efficient path form node 1 to node 6

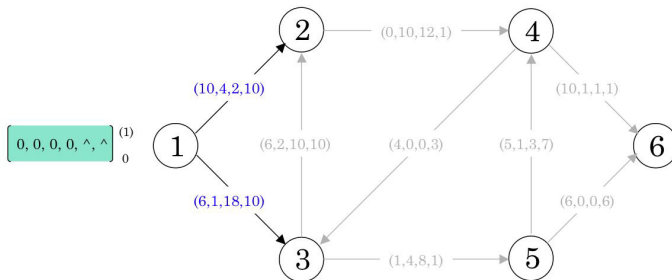
Initialization



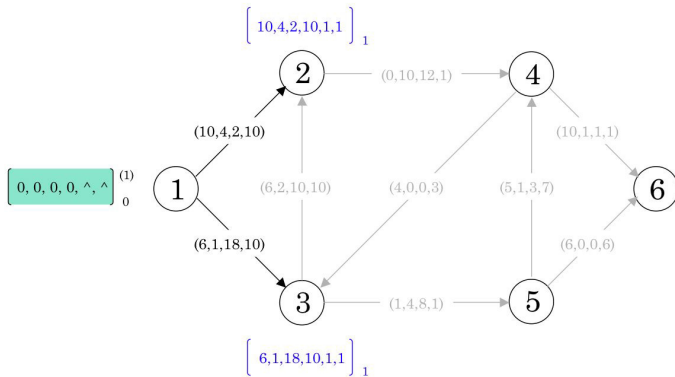
Iteration 1



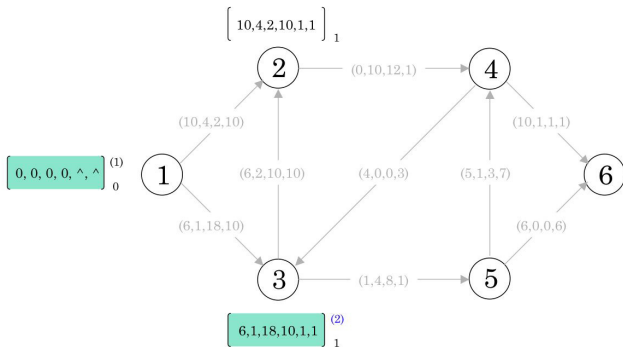
Iteration 1



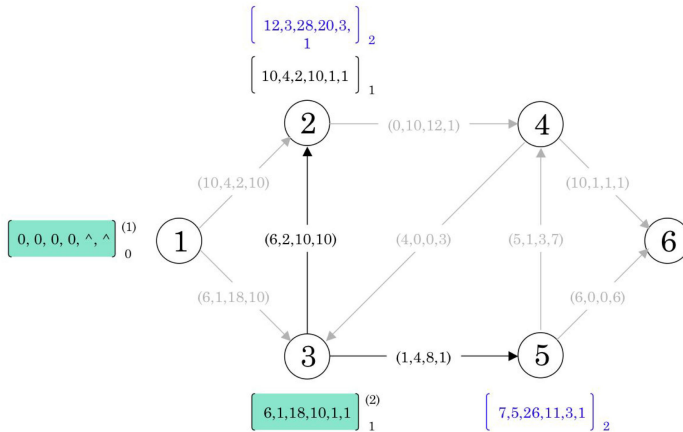
Iteration 1



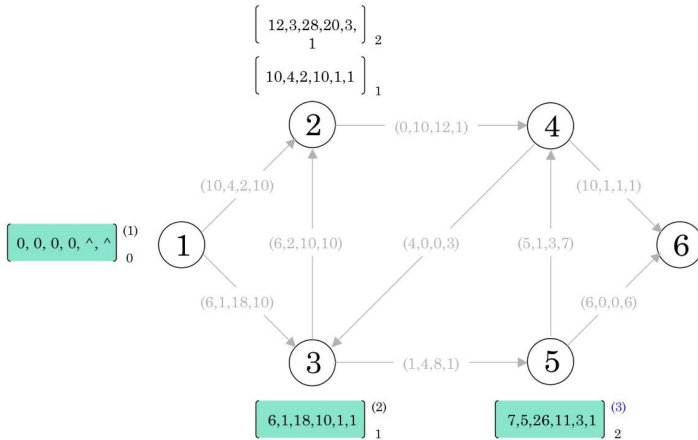
Iteration 2



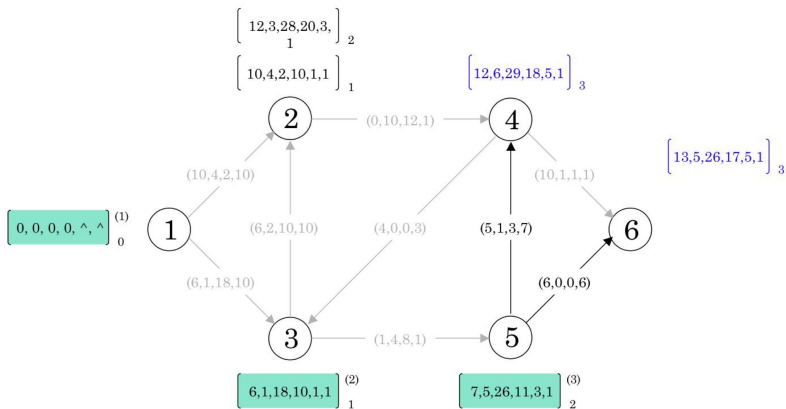
Iteration 2



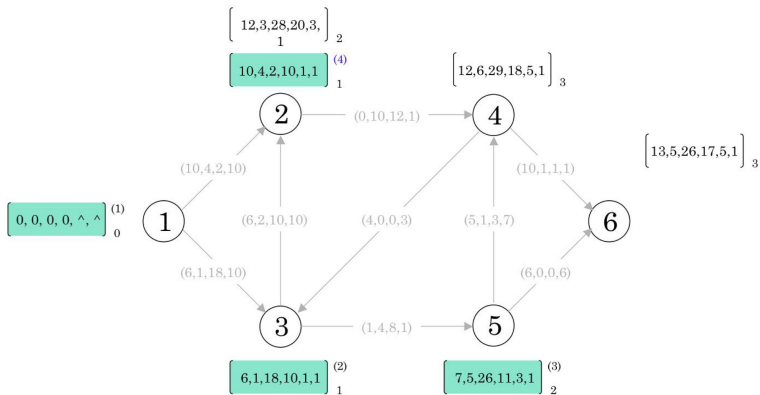
Iteration 3



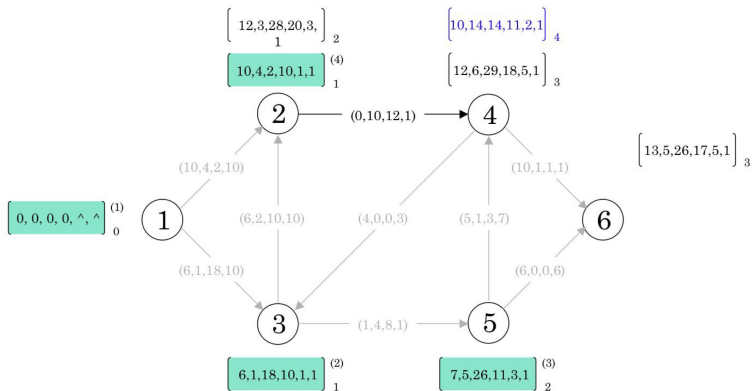
Iteration 3



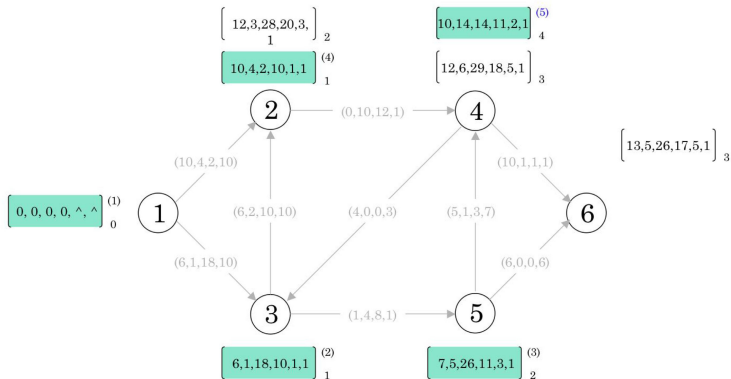
Iteration 4



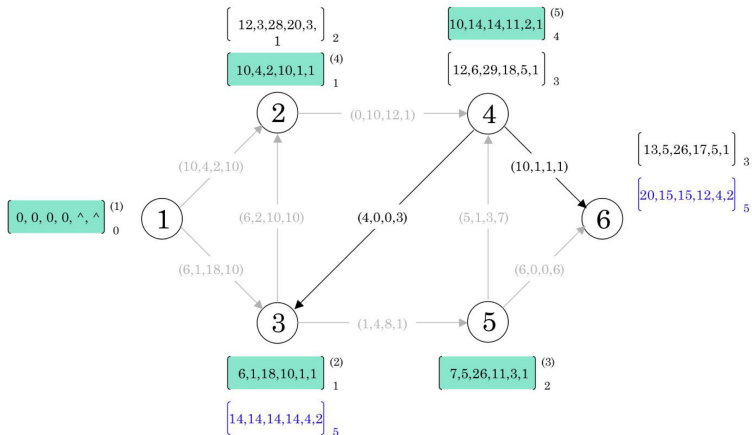
Iteration 4



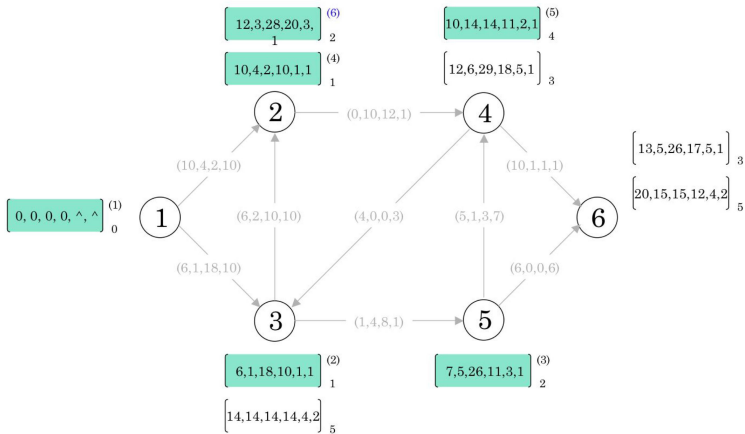
Iteration 5



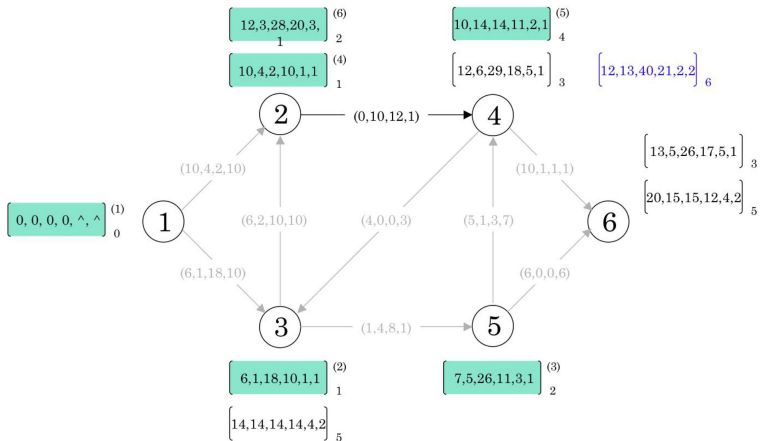
Iteration 5



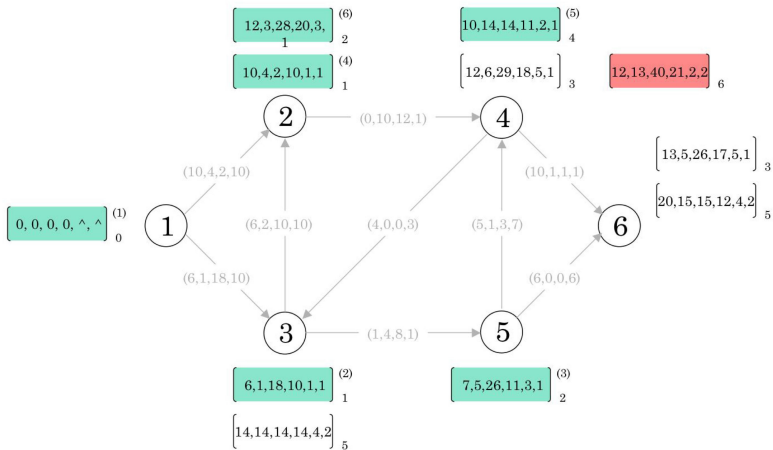
Iteration 6



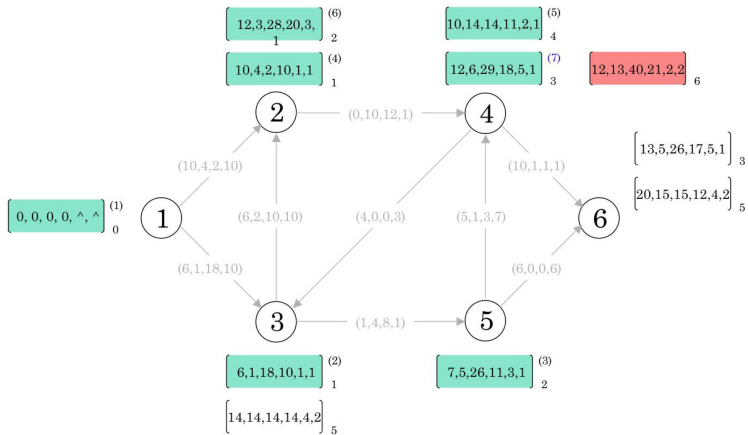
Iteration 6



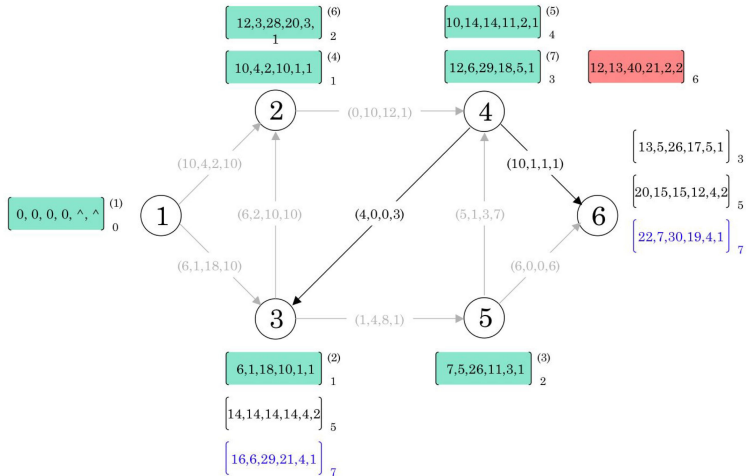
Iteration 6



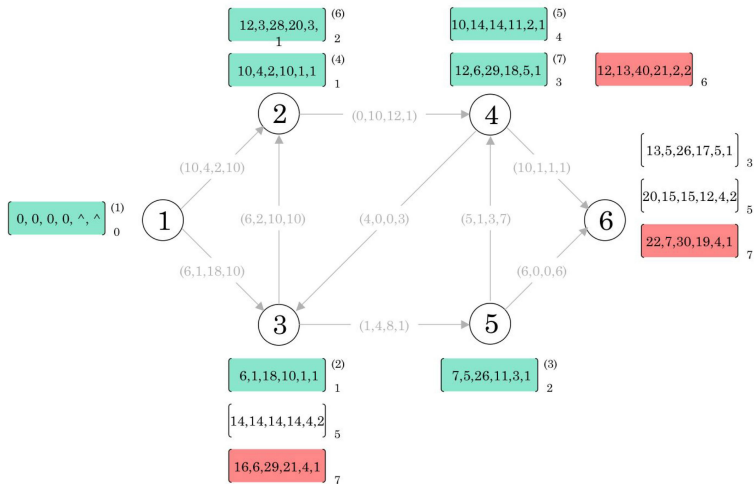
Iteration 7



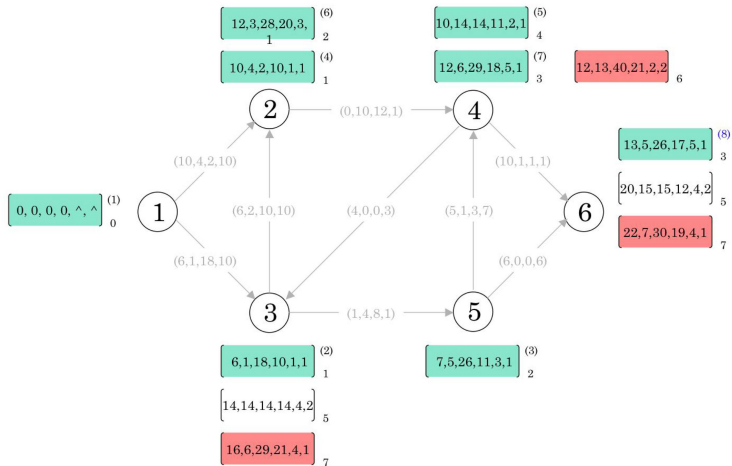
Iteration 7



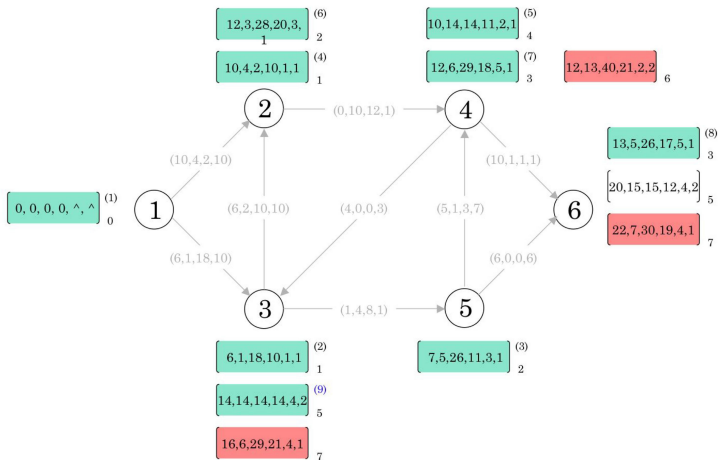
Iteration 7



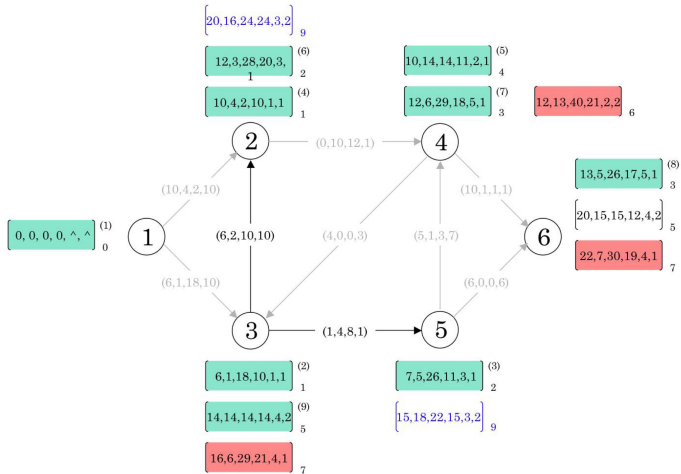
Iteration 8



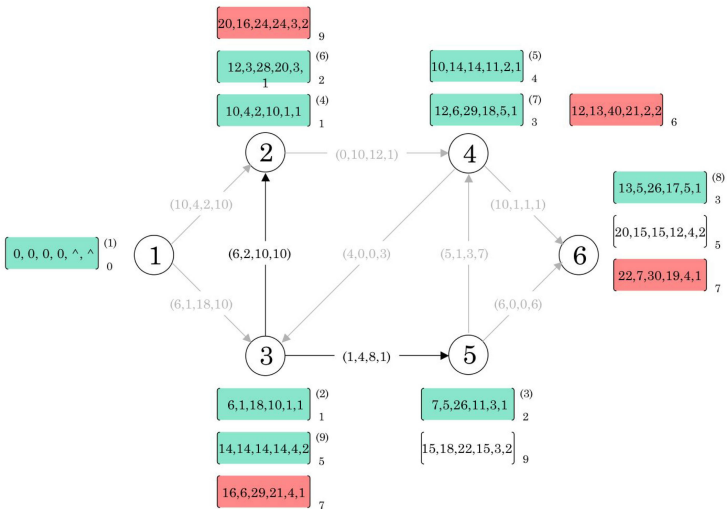
Iteration 9



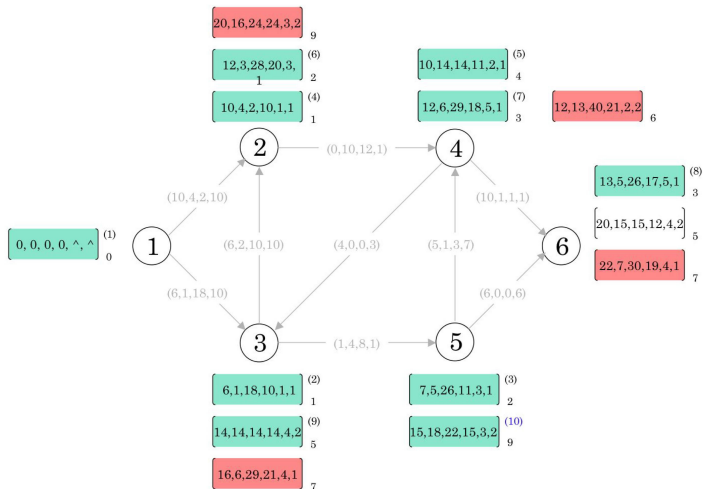
Iteration 9



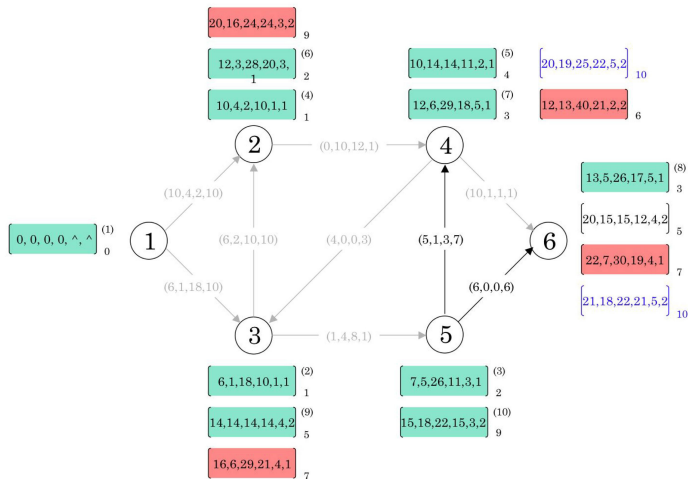
Iteration 9



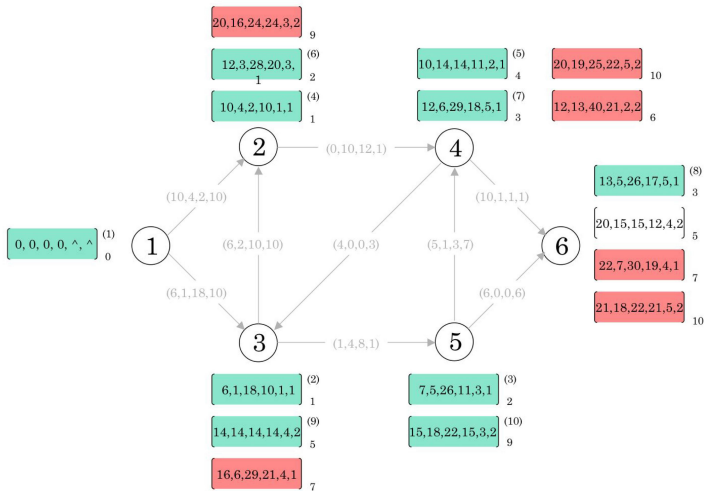
Iteration 10



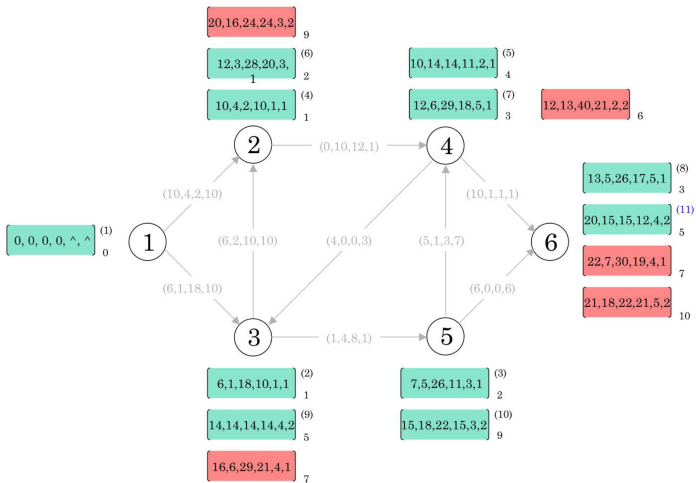
Iteration 10



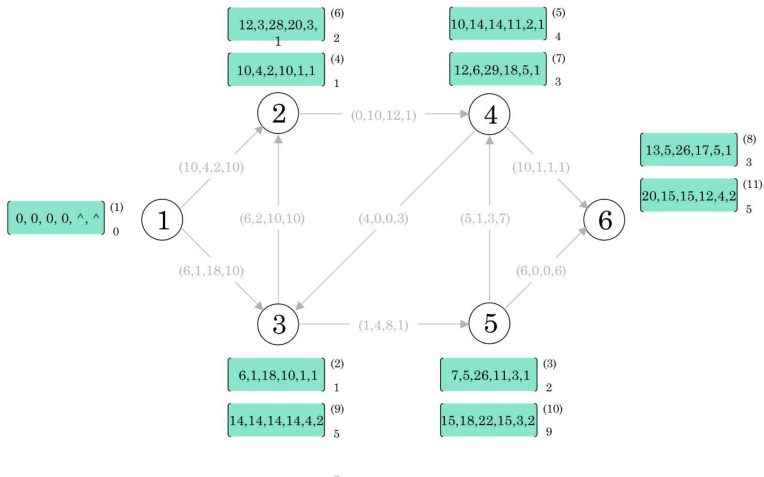
Iteration 10



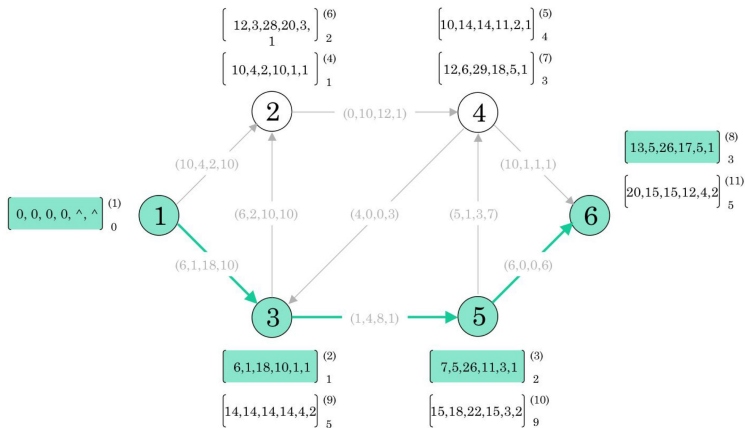
Iteration 11



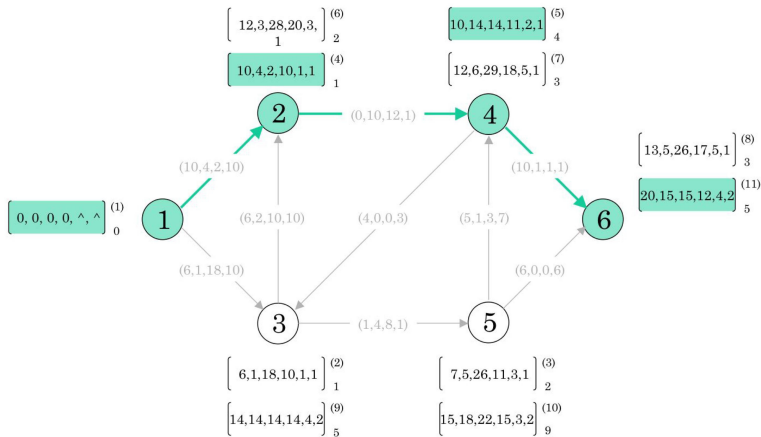
Result



Path 1



Path 2

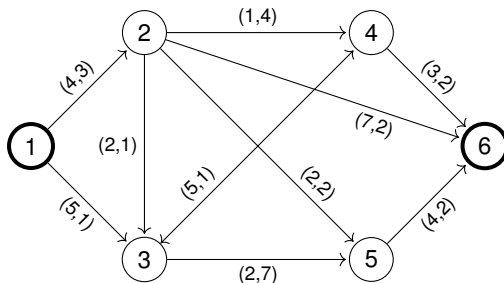


References and Pointers

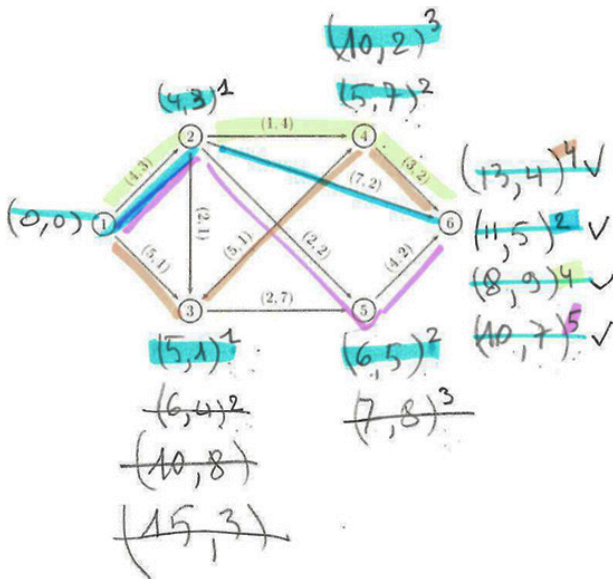
- ▶ P. Hansen. Bicriterion path problems. In: Fandel G, Gal T (eds) Multiple criteria decision making theory and application. *LNEMS*, 177. Springer, Berlin, pp. 109–127. 1979.
- ▶ E. Martins, On a multicriteria shortest path problem. *European Journal of Operational Research* 16:236–245, 1984.
- ▶ S. Randriamasy, X. Gandibleux, J. Figueira, and Ph. Thomin. Device and a method for determining routing paths in a communication network in the presence of selection attributes. Patent 11/25/04. #20040233850. Washington, DC, USA.
- ▶ X. Gandibleux, Fr. Beugnies, S. Randriamasy, Martins' algorithm revisited for multi-objective shortest path problems with a MaxMin cost function. *4OR: A Quarterly Journal of Operations Research*, Volume 4, Issue 1, Mar 2006, pp. 47-59.
- ▶ M. Iori, S. Martello, D. Pretolani. An aggregate label setting policy for the multi-objective shortest path problem. *European Journal of Operational Research* 207:1489–1496, 2010.
- ▶ C. Bornstein, N. Maculan, M. Pascoal, L. Pinto. Multiobjective combinatorial optimization problems with a cost and several bottleneck objective functions: An algorithm with reoptimization. *Computers & Operations Research* 39:1969–1976, 2012.

Exercise

Calculer Y_N du sommet 1 aux autres sommets du graphe

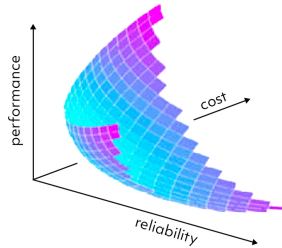


Solution



Multi-Objective Optimization

Software

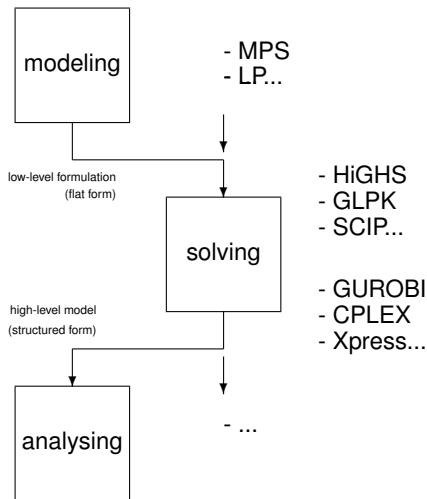


Native Multi-Objective Mathematical Programming Software

Multi-Objective Mathematical Programming Software

- JuMP
- GNU MP
- Pyomo...

- AMPL
- GAMS
- OPL...



Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- ▶ Algebraic Modeling Language (AML):

a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

- ▶ expressed in proprietary languages:

GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...

- ▶ close to programming languages:

JuMP (Julia), Pyomo (Python),...

- ▶ Solver-Oriented Interface (SOI):

tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

- ▶ Application Programming Interfaces (API):

API C++ Gurobi,...

- ▶ embedded Domain-Specific Languages (DSL) :

highspy, MATLAB, Hexaly Modeler,...

Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- ▶ Algebraic Modeling Language (AML):

a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

- ▶ expressed in proprietary languages:

GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...

- ▶ close to programming languages:

JuMP (Julia), Pyomo (Python),...

- ▶ Solver-Oriented Interface (SOI):

tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

- ▶ Application Programming Interfaces (API):

API C++ Gurobi,...

- ▶ embedded Domain-Specific Languages (DSL) :

highspy, MATLAB, Hexaly Modeler,...

Software for modeling activity

Modelers: software tools devoted to express mathematical optimisation problems

- ▶ **Algebraic Modeling Language (AML):**

a high-level programming language designed for mathematical optimisation with an emphasis on declarative modeling and independent of any particular solver.

- ▶ expressed in proprietary languages:

GAMS, AMPL, AIMMS, OPL, Mosel, GMP, ZIMPL, ...

- ▶ close to programming languages:

JuMP (Julia), Pyomo (Python),...

- ▶ **Solver-Oriented Interface (SOI):**

tightly coupled to a specific optimisation solver; provide programmatic control and access to solver-specific features within a host programming language (C/C++, Java, Python,...)

- ▶ Application Programming Interfaces (API):

API C++ Gurobi,...

- ▶ embedded Domain-Specific Languages (DSL) :

highspy, MATLAB, Hexaly Modeler,...

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- ▶ **Generic MILP solvers:**

implement numerical algorithms tailored to problem classes (LP, MIP,...)

- ▶ Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
- ▶ GLPK, SCIP, HiGHS, CBC/CLP,...

- ▶ **Specific MOMP solvers:**

specialized optimisation engines designed to directly handle problems with multiple objectives

- ▶ ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- ▶ **Generic MILP solvers:**

implement numerical algorithms tailored to problem classes (LP, MIP,...)

- ▶ Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
- ▶ GLPK, SCIP, HiGHS, CBC/CLP,...

- ▶ **Specific MOMP solvers:**

specialized optimisation engines designed to directly handle problems with multiple objectives

- ▶ ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

Software for solving activity

Optimisation solver: a specialized software engine designed to find solutions to mathematical programming problems

- ▶ **Generic MILP solvers:**

implement numerical algorithms tailored to problem classes (LP, MIP,...)

- ▶ Gurobi, CPLEX, Xpress, MOSEK, Hexaly, COPT,...
- ▶ GLPK, SCIP, HiGHS, CBC/CLP,...

- ▶ **Specific MOMP solvers:**

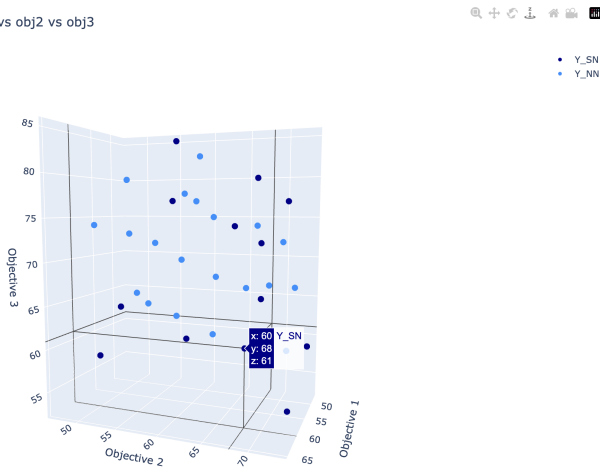
specialized optimisation engines designed to directly handle problems with multiple objectives

- ▶ ADBASE, BENSOLVE, INNER, PaMILO, PolySCIP, vOptSpecific

Software for analysing activity

Homemade...

Objective space : obj1 vs obj2 vs obj3



Summary of software examined



<i>Software</i>	<i>Commercial</i>	<i>Open-source</i>	<i>Modeler</i>	<i>Solver</i>	<i># Trade-offs</i>
MATLAB	✓	–	SOI	–	≥1
Gurobi Optimizer	✓	–	SOI	generic	1
HiGHS	–	✓	SOI	generic	1
COPT	✓	–	SOI	generic	1
Hexaly	✓	–	SOI	generic	1
FICO Xpress Optimizer	✓	–	AML+SOI	generic	1
CPLEX Optimisation Studio	✓	–	AML+SOI	generic	1
AMPL	✓	–	AML	–	1
AIMMS	✓	–	AML	–	1
GAMS	✓	–	AML	–	≥1
Pyomo	–	✓	AML	–	–
PuLP	–	✓	AML	–	–
CVXPY	–	✓	AML	specific	1
MOCVXPY	–	✓	AML	specific	≥1
BENSOLVE	–	✓	–	specific	≥1
INNER	–	✓	–	specific	≥1
PaMILO	–	✓	–	specific	≥1
SCIP with ZIMLP and PolySCIP	–	✓	AML	specific	≥1
JuMP with MultiObjectiveAlgorithms	–	✓	AML	meta-solver	≥1

JuMP with MultiObjectiveAlgorithms

Survey

Raise you hand if you are familiar with



Survey

Raise you hand if you are familiar with

- ▶ Matlab, Scilab or Mathematica
- ▶
- ▶



Survey

Raise you hand if you are familiar with

- ▶ Matlab, Scilab or Mathematica
- ▶ Python, or R
- ▶



Survey

Raise you hand if you are familiar with

- ▶ Matlab, Scilab or Mathematica
- ▶ Python, or R
- ▶ Fortran, C, or C++



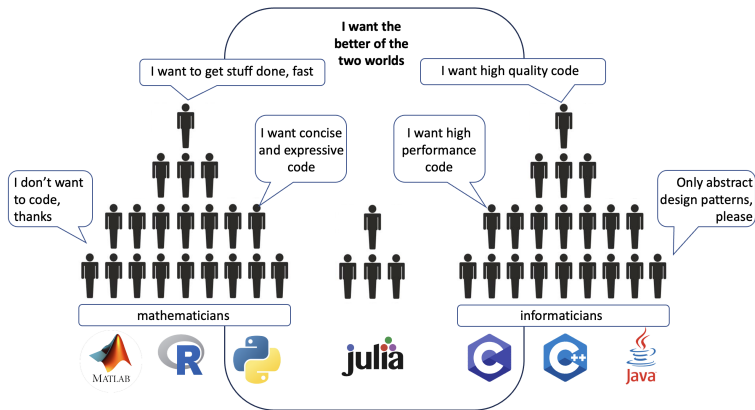
Raise you hand if you are familiar with

- ▶ Matlab, Scilab or Mathematica
- ▶ Python, or R
- ▶ Fortran, C, or C++



Maths formalisms as Matlab
Interactive as Python
Fast as C/C++

Why Julia and JuMP for MOO?

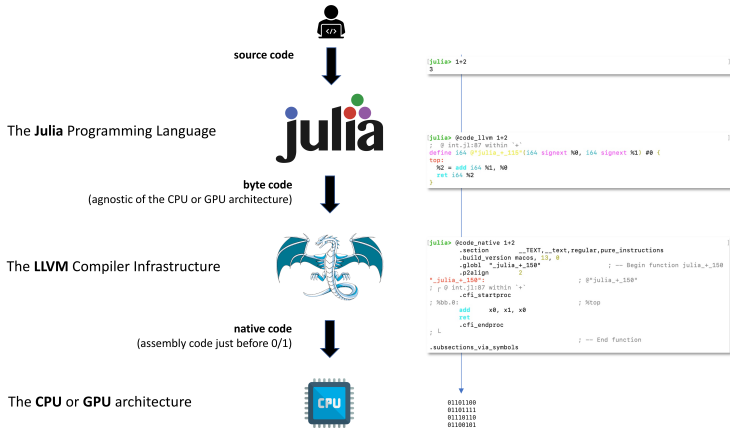


↳ **Julia** progr. language: easy, high-level, efficient, expressive

JuMP modeling language: expressive, efficient, evolutive

Efficient! Why?

LLVM: middleman between source code and compiled native code



Easy to use?



...the Julia REPL

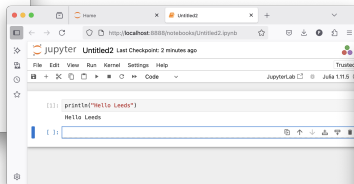
```
xavierg — Julia — julia — 80x24
Last login: Fri May 30 13:26:56 on ttys083
(base) xavierg@Xaviers-iMac ~ % julia

Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.11.5 (2025-06-14)
Official https://julialang.org/ release

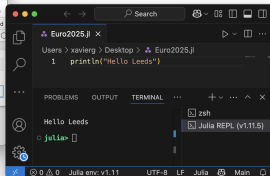
julia> println("Hello Leeds")
Hello Leeds

julia> 
```

...a notebook
(e.g. jupyter)



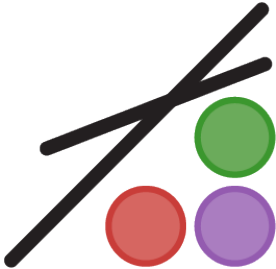
...an IDE
(e.g. VScode)



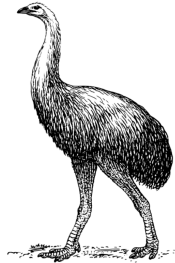
... or also in command line into a terminal.



JuMP with MultiObjectiveAlgorithms



version 1.30.0 and more

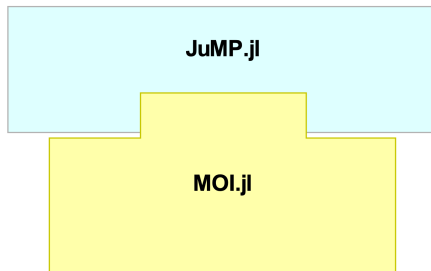


version 1.12.0 and more

Getting started with JuMP



```
julia> using Pkg  
julia> Pkg.add("JuMP")
```

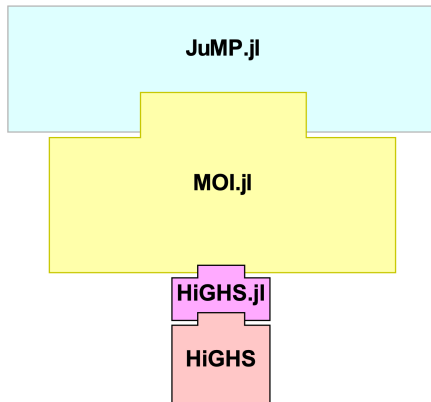


JuMP uses *MathOptInterface (MOI)*, an abstraction layer designed to provide a unified interface to mathematical optimization solvers.

Getting started

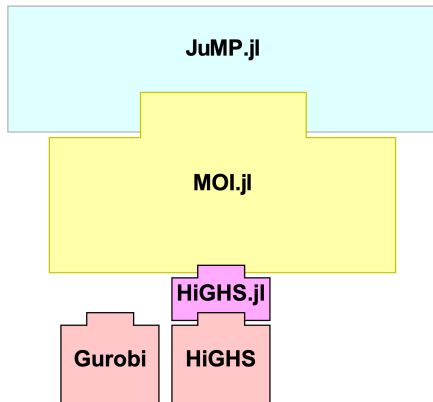


```
julia> Pkg.add("HiGHS")
```



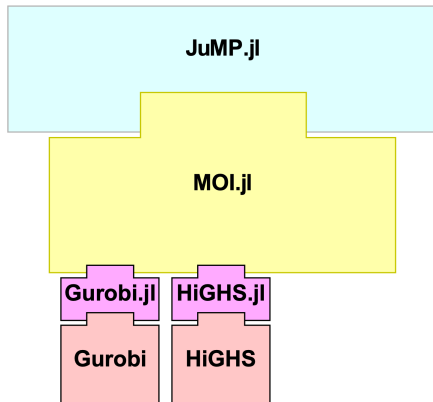
Getting started

With Gurobi or CPLEX or Xpress MP solver properly installed on your computer...



Getting started

```
julia> Pkg.add("Gurobi")
```



The MOO problem

Compute X_E and Y_N for the following MOO problem:

$$\begin{array}{llllll} \max z_1 & = & x_1 & + & x_2 & \\ \min z_2 & = & x_1 & + & 3x_2 & \\ s.t. & & 2x_1 & + & 3x_2 & \leq 30 \\ & & 3x_1 & + & 2x_2 & \leq 30 \\ & & x_1 & - & x_2 & \leq 5.5 \\ & & x_1 & , & x_2 & \in \mathbb{N} \end{array}$$



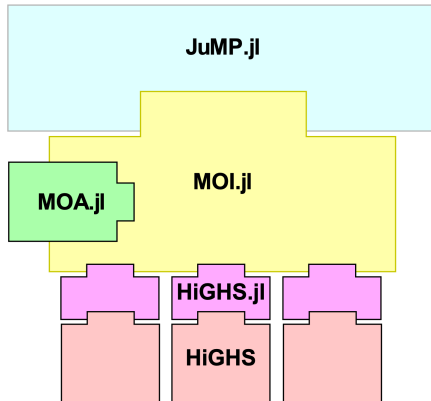
Installing the package:

```
julia> using Pkg
```

```
julia> Pkg.status()  
Status `~/julia/environments/v1.11/Project.toml`  
 [60bf3e95] HiGHS v1.18.1  
 [4076af6c] JuMP v1.26.0
```

```
julia> Pkg.add("MultiObjectiveAlgorithms")
```

Getting Started (con't)



Algorithms currently available (v1.11.0)

- | | |
|----------------------------------|------------|
| 1. MOA.Lexicographic() [default] | $p \geq 2$ |
| 2. MOA.Dichotomy() | $p = 2$ |
| 3. MOA.EpsilonConstraint() | $p = 2$ |
| 4. MOA.Hierarchical() | $p \geq 2$ |
| 5. MOA.Chalmet() | $p = 2$ |
| 6. MOA.KirlikSayin() | $p \geq 2$ |
| 7. MOA.DominguezRios() | $p \geq 2$ |
| 8. MOA.TambyVanderpooten() | $p \geq 2$ |
| 9. MOA.GeneralDichotomy() | $p \geq 2$ |

+ optimization attributes coming with a given algorithm

Algorithms currently available (v1.11.0)

- | | |
|----------------------------------|------------|
| 1. MOA.Lexicographic() [default] | $p \geq 2$ |
| 2. MOA.Dichotomy() | $p = 2$ |
| 3. MOA.EpsilonConstraint() | $p = 2$ |
| 4. MOA.Hierarchical() | $p \geq 2$ |
| 5. MOA.Chalmet() | $p = 2$ |
| 6. MOA.KirlikSayin() | $p \geq 2$ |
| 7. MOA.DominguezRios() | $p \geq 2$ |
| 8. MOA.TambyVanderpooten() | $p \geq 2$ |
| 9. MOA.GeneralDichotomy() | $p \geq 2$ |

+ optimization attributes coming with a given algorithm

Algorithms currently available (v1.11.0)

1. MOA.Lexicographic() [default]	$p \geq 2$
2. MOA.Dichotomy()	$p = 2$
3. MOA.EpsilonConstraint()	$p = 2$
4. MOA.Hierarchical()	$p \geq 2$
5. MOA.Chalmet()	$p = 2$
6. MOA.KirlikSayin()	$p \geq 2$
7. MOA.DominguezRios()	$p \geq 2$
8. MOA.TambyVanderpooten()	$p \geq 2$
9. MOA.GeneralDichotomy()	$p \geq 2$

+ optimization attributes coming with a given algorithm

Zoom on the ϵ -constraint algorithm

Algorithm:

▶ `EpsilonConstraint()`

Y.V. Haimes, L.S. Lasdon, D.A. Wismer (1971). On a bicriterion formation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man and Cybernetics*, Volume SMC-1, Issue 3, Pages 296-297.

Attributes:

▶ `MOA.EpsilonConstraintStep()`

algorithm uses this value as the ϵ by which it partitions the first-objective's space. The default is 1, so that for a pure integer program this algorithm will enumerate Y_N .

▶ `MOA.SolutionLimit()`

if this attribute is set then, instead of using the `MOA.EpsilonConstraintStep`, with a slight abuse of notation, `EpsilonConstraint` divides the range of the first-objective's domain in objective space by `SolutionLimit` to obtain the ϵ to use when iterating. Thus, at most `SolutionLimit` solutions are returned.

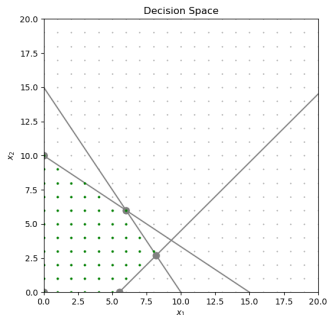
Example (1/4)



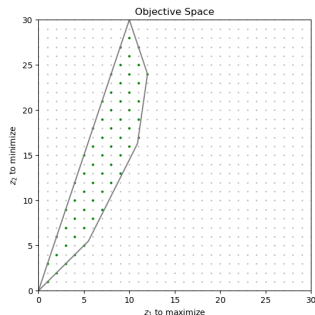
$$\begin{aligned} \max z_1 &= x_1 + x_2 \\ \min z_2 &= x_1 + 3x_2 \\ \text{s.t.} \quad &2x_1 + 3x_2 \leq 30 \\ &3x_1 + 2x_2 \leq 30 \\ &x_1 - x_2 \leq 5.5 \\ &x_1, x_2 \in \mathbb{N} \end{aligned}$$

```
julia> c1 = [1,1]
julia> c2 = [1,3]
julia> A = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
```

Decision space:



Objective space:





Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1 ≥ 0, Int)
julia> @variable(model, x2 ≥ 0, Int)
julia> @expression(model, fct1, x1 + x2) # to maximize
julia> @expression(model, fct2, x1 + 3 * x2) # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ≤ 30)
julia> @constraint(model, 3*x1 + 2*x2 ≤ 30)
julia> @constraint(model, x1 - x2 ≤ 5.5)
```



Coding the MOO problem with JuMP

```
julia> using JuMP, HiGHS
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1 ≥ 0, Int)
julia> @variable(model, x2 ≥ 0, Int)
julia> @expression(model, fct1, x1 + x2)           # to maximize
julia> @expression(model, fct2, x1 + 3 * x2)       # to minimize
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ≤ 30)
julia> @constraint(model, 3*x1 + 2*x2 ≤ 30)
julia> @constraint(model, x1 - x2 ≤ 5.5)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

```
julia> optimize!(model)
```



Setup the MIP solver: e.g. HiGHS

```
julia> set_optimizer(model, ()->MOA.Optimizer(HiGHS.Optimizer))
```

Setup the algorithm: ϵ -constraint; step=1 (default value)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

Optimize the MOO problem

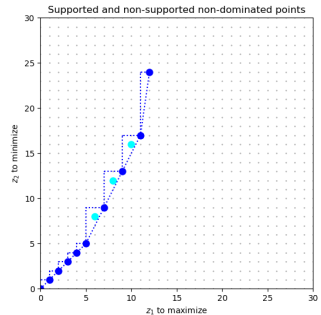
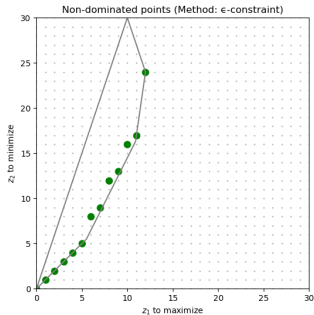
```
julia> optimize!(model)
```

Example (4/4)



Get X_E and Y_N

```
julia> for i in 1:result_count(model)
julia>     z1_opt = objective_value(model; result = i)[1]
julia>     z2_opt = -1 * objective_value(model; result = i)[2]
julia>     x1_opt = value(x1; result = i)
julia>     x2_opt = value(x2; result = i)
julia> end
```



Key Takeaways...

You are now able to...

- ▶ identify existing software for multi-objective optimization
- ▶ formulate a multi-objective (linear) optimization model
- ▶ implement and solve it in *Julia* using *JuMP* and *MOA*
- ▶ extract and interpret X_E and Y_N produced by the solver
- ▶ choose an appropriate solution algorithm for a given problem

... use *Julia*, *JuMP* and *MOA* as a integrated workflow for multi-objective optimization

Next Steps...

Going further with JuMP:

- ▶ modify the model data, the model itself
- ▶ advanced modeling techniques, including sets and indices
- ▶ advanced resolution techniques, including callbacks to solvers
- ▶ other classes of optimization problems

Going further with MOA:

- ▶ compare the MOA algorithms with your own algorithm
- ▶ contribute by integrating your own algorithm into MOA

Going further with Julia:

- ▶ discover and learn to code in Julia
- ▶ use packages from among the 12,000 available packages

Références...

JuMP (documentation, resources, community, etc.):

<https://jump.dev/>
<https://jump.dev/JuMP.jl/stable/>

Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. 2023.

MultiObjectiveAlgorithms (source code, examples, tutorials):

<https://github.com/jump-dev/MultiObjectiveAlgorithms.jl>
https://jump.dev/JuMP.jl/stable/tutorials/linear/multi_objective_examples/

Oscar Dowson, Xavier Gandibleux, Gökhan Kof. MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems. *INFORMS Journal of Computing*, 2026.

Xavier Gandibleux. Native Multi-Objective Mathematical Programming Software. Chapter 15 in *Handbook on Multi-objective Combinatorial Optimisation* (M. Ehrgott, J. Figueira, Ö. Karsu, L. Paquete Edts.). Springer, 2026 (to appear).

Example

For the bi-objective unidimensional 01 knapsack problem,

$$\max \{ (c^1 x, c^2 x) \mid wx \leq \omega, x \in \{0, 1\}^n \}$$

with¹

$$n = 6$$

$$c^1 = (2, 2, 5, 9, 8, 6)$$

$$c^2 = (8, 2, 6, 2, 5, 8)$$

$$w = (8, 8, 7, 5, 4, 2)$$

$$\omega = 17$$

compute the lexicographic optimal solutions.

¹Example taken from the PhD thesis of Julien Jorge. *Nouvelles propositions pour la résolution exacte du sac à dos multi-objectif unidimensionnel en variables binaires*. PhD Thesis (in French). Université de Nantes, France, 2010.

Setup the data

```
julia> c1 = [2,2,5,9,8,6]           # coef vct of the obj 1
julia> c2 = [8,2,6,2,5,8]           # coef vct of the obj 2
julia> w = [8,8,7,5,4,2]            # coef vct of weights
julia>  $\omega$  = 17                    # nominal capacity
julia> n = length(c1)                # number of items
```

Setup the model

```
julia> kp = Model( )  
julia> @variable(kp, x[1:n], Bin)  
julia> @expression(kp, fct1, sum(c1[j]*x[j] for j=1:n))  
julia> @expression(kp, fct2, sum(c2[j]*x[j] for j=1:n))  
julia> @objective(kp, Max, [fct1, fct2])  
julia> @constraint(kp, sum(w[j]*x[j] for j=1:n) ≤  $\omega$ )
```

Setup the solver and the algorithm to apply

```
julia> set_optimizer(kp,()->MOA.Optimizer(GLPK.Optimizer))
```

```
julia> set_attribute(kp, MOA.Algorithm(),  
                    MOA.Lexicographic() )
```

Solve and display results (1/2)

Invoking the solver:

```
julia> optimize!(kp)
```

Summary of the resolution:

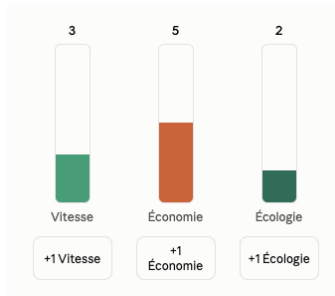
```
julia> solution_summary(kp)
```

Displaying the results (X_E and Y_N):

```
julia> for i in 1:result_count(kp)
julia>     println( i )
julia>     println( objective_value(kp; result = i) )
julia>     println( value.(x; result = i) )
julia> end
```

Multi-Objective Optimization

Case studies



the bi-objective 01 Uncapacitated Facility Location Problem

the bi-objective Truck-to-dock Door Assignment Problem